Final Year Project

Notes House

By

Hasnat Ahmad 2022-GCG-123

Abdul Hanan 2022-GCG-126

Hafiz Abdul Rahman 2022-GCG-149

Under the supervision of

Prof. Kashif Riaz

Bachelor of Science in Information Technology

(2022-2026)

FACULTY OF COMPUTING & INFORMATION TECHNOLOGY (FCIT),

UNIVERSITY OF THE PUNJAB, LAHORE.

Notes House

A project presented to

University of the Punjab, Lahore

In partial fulfilment of the requirement for the degree of

Bachelor of Science in Information Technology

(2022-2026)

By

Hasnat Ahmad 2022-GCG-123

Abdul Hanan 2022-GCG-126

Hafiz Abdul Rahman 2022-GCG-149

FACULTY OF COMPUTING & INFORMATION TECHNOLOGY (FCIT),

UNIVERSITY OF THE PUNJAB, LAHORE.

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Signature: _____

Hasnat Ahmad: 076421

Signature:_____

Abdul Hanan: 076329

Signature:_____

Hafiz Abdul Rahman: 076370

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BS-IT "Notes House" was developed by Hasnat Ahmad (2022-GCG-123)**,** Abdul Hanan (2022-GCG-126) and Hafiz Abdul Rahman (2022-GCG-149) under the supervision of "Prof. Kashif" in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Information Technology.

Signature:_____

FYDP Supervisor

Signatures (Faculty Advisory Committee (FAC)

Signatures			
Name			
	FAC1	FAC2	FAC3

Signature: _____

Head of FYDP Coordination Office:

Signature: _____ Date:_____

Chairperson, department of [Information Technology]

Executive Summary

Notes House is a mobile application developed to help university students access study materials in one organized platform. Students can view notes, past papers, and outlines by semester and subject. The app also includes an in-app PDF reader and an AI chatbot that allows students to ask questions related to their study material and receive helpful responses instantly.

Notes House also provides a quiz system to test preparation and analyze performance through results and graphs. Teachers can upload study materials, which are verified by admins before becoming visible to students. The system ensures secure access, easy navigation, and a smooth learning experience for all users.

Keywords: Study Material, AI Chatbot, Quiz System, Teacher Upload, Admin Approval

Acknowledged

The acknowledgment section is an opportunity to express gratitude and appreciation to individuals and organizations who have contributed to the completion of your Final Year Project (FYP).

*Signature:*_____

Hasnat Ahmad: 076421

*Signature:*_____

Abdul Hanan: 076329

*Signature:*_____

Hafiz Abdul Rahman: 076370

Abbreviations

Table of all the abbreviations and acronyms used in your FYP along with their respective brief description. The abbreviation list must be ordered alphabetically.

PDF	Portable Document Format- A file format used to display documents in a consistent layout across devices.
Edu	Education- The process of acquiring knowledge, skills, and training through learning and instruction
OS	Operating System- The process of acquiring knowledge, skills, and training through learning and instruction
AI	Artificial Intelligence- Technology that enables machines to simulate human intelligence and provide automated responses.
API	Application Programming Interface- A set of rules that allows different software components to communicate with each other.
SDK	Software Development Kit- A collection of tools and libraries used to develop applications.
JDK	Java Development Kit-A development environment used to build Java-based applications.
iOS	iPhone Operating System- A mobile operating system developed for Apple devices.
LTS	Long Term Support- A software version that receives extended updates and security fixes.
LLM	Large Language Model- An AI model trained on large datasets to understand and generate text.
Q/A	Question and Answer-A format of communication where a question is asked and a corresponding response is provided to clarify information or solve a problem.
SRS	Software Requirements Specification- A document that defines system requirements and functionality.
UI/UX	User Interface / User Experience- The visual design and overall user interaction experience of an application.
FYP	Final Year Project- The final academic project completed in the last year of a degree program.
ES	Early Start- Software that manages hardware resources and provides services for applications.
EF	Early Finish- The earliest time an activity can be completed based on its start time and duration.
LS	Late Start- The earliest time an activity can be completed based on its start time and duration.
LF	Late Finish- The latest time an activity can be completed without affecting project completion.

TS	Total Slack- The amount of time an activity can be delayed without delaying the project end date.
FS	Free Slack- The amount of time an activity can be delayed without affecting the next activity.
UC	Use Cases- A description of how users interact with a system to achieve specific goals.
FR	Functional Requirements- Specifications that describe what the system should do and the features it must provide.
NFR	Non-Functional Requirements- Specifications that define system performance, security, usability, and other quality attributes.
DOC	Document- A written or digital file that records information for reference or communication.
AES	Advanced Encryption System- A secure encryption method used to protect data.
GUI	Graphical User Interface- A visual interface that allows users to interact with software using icons, buttons, and menus.
FAQ	Frequently Asked Questions- A section that provides answers to common user queries.
FCM	Firebase Cloud Messaging- A service used to send push notifications to mobile applications.
HTTPS	Hyper Text Transfer Protocol Secure - A secure protocol used for encrypted communication over the internet.

Table of contents

List of Figures

List of Tables

Chapter 1

Introduction

1. Introduction

This chapter gives an overview of our project titled **Notes House**. Students from different universities in Pakistan often face problems while searching for notes, past papers, and course outlines. Most study material is found in random WhatsApp groups, old drives, or scattered across social media. Because of this,students waste alot of time searching instead of studying.Notes House provides a simple and organized solution by collecting all study materials in one place. It helps both students and teachers by offering easy access, secure uploads, and verified notes for better learning.

1.1 Problem Statement

Students in Pakistan often face difficulty while searching for proper study material because notes, past papers, and course outlines are scattered on different WhatsApp groups, or saved by seniors. As a result, students waste valuable time trying to find the right material instead of focusing on their studies.

Many students also do not know if the notes they find are correct, complete, or updated, which creates confusion and stress during exams. In most cases, the material available online is unorganized and of poor quality, making it hard for students to trust it. Teachers also suffer from the lack of a proper platform where they can upload and share their notes with students in a safe and reliable way. Even when teachers want to help, they do not have one secure system that allows them to share verified study resources with everyone.

Students further face problems because they do not get organized quizzes or tools to check their preparation level. Without proper practice and analysis, they cannot understand their weak areas before exams. Students use various chatbots to clear their topics concepts which is time consuming. Due to all these issues, both students and teachers need a single platform where study material is complete, verified, and easy to access. This system will make studying easier, more reliable, and stress-free for students across Pakistan.

1.2 Problem Solution

Notes House is a mobile app that helps students easily find and access all their study material in one place. Instead of searching through WhatsApp groups or asking seniors for notes, students can quickly view verified notes, past papers, and course outlines inside the app. The main goal of the app is to save students' time and reduce the stress they face during exam preparation.

Teachers can also upload their notes, and after verification, the material becomes available for all students. This ensures that every student gets high-quality and authentic study content. The app includes a built-in PDF reader, so students do not need any third-party apps to open their files. Notes House also allows students to download study material inside the app, but the files stay protected and cannot be shared outside. To help students test their preparation, the app provides quizzes for different subjects and stores the results for future analysis. Students can view their performance through simple graphs to understand their strong and weak areas.

Notes House also includes an AI-based chat feature where users can ask questions about any document they are reading. This helps students understand difficult topics quickly and easily. The app's interface is simple and user-friendly, making it suitable for students from all universities. Admins manage the uploaded content to ensure the platform stays clean and organized. With all these features combined, Notes House provides a complete solution to the problem of unorganized and unreliable study material.

1.3 Objectives of the Proposed System

Centralized Study Material:

Provide all notes, past papers, and course outlines in one place to save students' time.

Verified Teacher Uploads:

Allow teachers to upload study material after verification to ensure high-quality and authentic content.

Admin Approval System:

Make sure all uploaded notes are checked and approved by admins before becoming visible to students.

Built-in PDF Reader:

Give students the ability to read files inside the app without needing any third-party applications.

In-App Protected Downloads:

Allow students to download files inside the app while keeping the files safe from being shared outside.

Quiz Practice Feature:

Provide quizzes for different subjects to help students test their knowledge and prepare for exams.

Performance Analytics:

Show simple reports and graphs of quiz results so students can understand their strong and weak areas.

AI-Based Q/A Support:

Help students understand difficult topics by allowing them to ask questions about any document inside the app.

Easy App Navigation:

Make the app simple and user-friendly so students can find their required material quickly.

Secure Data Storage:

Keep all files and user data protected with secure storage to prevent unauthorized access.

Reliable App Performance:

Ensure the app works smoothly with high uptime so students and teachers can use it anytime.

Multi-University Support:

Support study material from all universities across Pakistan in an organized and structured way.

Teacher-Student Interaction (Indirect):

Provide a clean way for teachers to share their knowledge with students through verified uploads.

Study Stress Reduction:

Reduce students' exam stress by giving them quick access to complete and trustworthy study resources.

1.4 Scope

1. Providing students with organized notes, past papers, and course outlines for different universities.
2. Allowing teachers to upload study material after verification through their edu email.
3. Enabling admins to check, approve, or reject uploaded content to maintain quality.
4. Including a built-in PDF reader so users can read documents inside the app.
5. Allowing protected in-app downloads while preventing sharing files outside the app.
6. Offering quizzes and basic performance reports to help students test their preparation.
7. Providing an AI chat feature that answers questions from the documents available in the app.
8. Ensuring secure storage of files and user data with proper access control.
9. Supporting a simple and user-friendly interface for quick navigation.

1.5 System Components – Notes House

Notes House is an educational mobile application with three main parts: **Student App**, **Teacher App**, and **Admin Panel**. Each part contains different modules that work together to provide easy access to study material, quizzes, uploads, and secure data handling. Below are the detailed components:

1.5.1Student Side App (NotesHouse Mobile Application)

1. Study Material Access Module:

1 Allows students to view semester-wise notes, past papers, and course outlines.
2 Provides search and filter options to find required material quickly.
3 Displays only approved and verified content uploaded by teachers.

2. PDF Reader & In-App Download Module:

1 Opens notes and past papers inside the built-in PDF reader.
2 Allows protected in-app downloads that cannot be shared outside.

3 Supports zoom, scroll, and basic reading controls for user comfort.

3. Quiz & Analysis Module:

1 Provides subject-wise quizzes for students to practice.
2 Stores quiz scores and attempts in the database.
3 Shows simple performance graphs to highlight strengths and weaknesses.

4. AI Chat Module:

1 Allows students to ask questions related to the document they are reading.
2 Provides AI-generated answers based on the content available inside the app.
3 Helps students understand difficult topics easily and quickly.

5. Profile & Settings Module:

1 Allows students to update profile details such as name and email.
2 Enables notification settings to turn alerts ON or OFF.
3 Displays downloaded files, quiz history, and app information.

1.5.2 Teacher Side App (Notes House Mobile App)

1. Teacher Verification Module:

1. Allows teachers to register using their official edu email.
2. Sends verification link to confirm teacher identity.
3. Activates teacher features only after verification is successful.

2. Upload Material Module:

1. Allows teachers to upload notes, outlines, and past papers.
2. Lets teachers add details like university, semester, subject, and file type.
3. Sends uploaded material to the admin for approval before publishing.

3. Quiz Creation Module:

1. Allows teachers to create subject-wise quizzes for students.
2. Enables editing or deleting quizzes when needed.
3. Submits quizzes for admin review before they appear to students.

4. Upload Tracking Module:

1. Shows teachers the status of their uploaded content (Pending, Approved, Rejected).
2. Displays admin feedback for corrections or improvements.

1.5.3 Admin Side (Notes House Mobile App)

5. Content Approval Module:

1. Allows admins to review uploaded notes, outlines, and past papers.
2. Enables approving, rejecting, or requesting changes from teachers.
3. Ensures only high-quality and correct material is published.

6. Teacher Management Module:

1. Manages verified teacher accounts and handles approval for new teachers.
2. Blocks or suspends teachers who upload inappropriate or low-quality content.

7. Quiz Management Module:

1. Allows admins to review quizzes created by teachers.
2. Approves or rejects quizzes to maintain consistency and correctness.

1.6 Related System Analysis/Literature Review

Table 1- 1 Related System Analysis with proposed project solution

Application Name	Weakness	Proposed Project Solution
Past Papers Pakistan	Only provides past papers, no notes or course outlines. Content is not verified, and some papers are outdated.	Notes House offers notes, outlines, and past papers with proper semester-wise structure. All uploads are verified by admins.
Ilmkidunya – Past Papers App	Focuses mainly on school/college past papers and basic notes. Does not cover university semesters, nor allow teacher uploads.	Notes House covers **all Pakistani universities**, supports teacher uploads, and includes complete semester-wise study material.
Studocu – Study Notes Sharing App	Many documents are locked behind premium subscription. Quality varies because anyone can upload without verification.	Notes House keeps all content **free**, accepts only teacher-approved uploads, and requires admin approval for quality control.
All Past Papers – Offline	Provides only past papers; no notes, outlines, or quizzes. Missing a built-in PDF reader and content verification.	Notes House includes **notes + past papers + outlines + quizzes**, along with a built-in PDF reader and secure in-app downloads.

1.7 Vision Statement

For university students across Pakistan

Who need an easy, organized, and reliable way to access notes, past papers, outlines, quizzes, and verified study material in one place.

The Notes House app

Is a complete educational learning platform

That provides verified notes, semester-wise past papers, protected in-app downloads, a built-in PDF reader, quizzes with analysis, and an AI chat system to help students understand difficult topics.

Unlike existing study apps that only offer past papers, unverified notes, or paid content, and do not support teacher uploads or admin approvals.

Our product offers a fully verified, free, and secure academic resource hub with teacher uploads, admin moderation, AI support, and a clean user-friendly interface designed specifically for university students.

1.8 System Limitations and Constraints

Limitations:

1. **Device Compatibility:**
   Notes House requires Android OS 8.0 or above, which may limit access for users with very old devices.
2. **Internet Dependency:**
   Accessing notes, quizzes, and AI chat features requires an internet connection. Students in remote areas may face difficulty using all features.
3. **Content Availability:**
   The system depends on teachers and admins to upload and approve material. Some universities or semesters may initially have limited content.
4. **Data Accuracy:**
   Notes and past papers provided by teachers may sometimes contain mistakes. The system relies on teacher accuracy and admin review.
5. **Limited Offline Features:**
   Only downloaded files are available offline. Other features like search, quizzes, and AI chat require online access.

Constraints:

1. **Scope of the Project:**
   The current scope includes notes, past papers, outlines, quizzes, in-app downloads, teacher uploads, and an AI chat. Features like live classes or video lectures are not included.
2. **Development Timeline:**
   The project must be completed within the academic year, limiting the addition of advanced features such as dynamic AI-generated quizzes or in-depth analytics.

3. **Testing Constraints:**
   Testing is focused mainly on Android devices. iOS testing is limited due to device availability and cost.

4. **Storage and Server Limits:**
   Cloud storage and database usage must stay within student project budget limits, affecting the number of large files stored.

5. **AI Integration Cost:**
   Advanced AI features may require paid APIs, so only essential AI capabilities are included in the initial version.

1.9 Tools and Technologies

Mention all the hardware/software tools and technologies with version information used in implementation of the project. Write about the APIs, language(s), SDK(s) etc. which you will use for implementation.

Table 1-2 Tools and Technologies for Proposed Project

Tools And Technologies	**Tools**	**Version**	**Rationale**
	Flutter SDK	3.x	Used to build a cross-platform mobile app (Android/iOS) with a single codebase.
	Android Studio	Latest Stable	Primary IDE for developing, testing, and debugging the mobile application.
	Firebase Console	Latest	Used for backend management such as authentication, storage, and database services.
	VS Code	Latest	Lightweight editor for writing Flutter code quickly and efficiently.
	Technology	**Version**	**Rationale**
	Dart Programming Language	Latest Stable	The programming language used by Flutter to develop the app's frontend.
	Firebase Authentication	Latest	Handles secure login for Students, Teachers, and Admins.
	Firebase Firestore Database	Latest	Stores notes metadata, user profiles, quiz data, and teacher uploads.

	Firebase Storage	Latest	Stores uploaded notes, past papers, outlines, and PDF files securely.
	Firebase Cloud Functions	Latest	Used for teacher verification, admin approval workflows, and handling AI chatbot backend tasks.
	RESTful APIs / Node.js (optional)	Latest LTS	For implementing backend logic if additional custom server-side work is required.
	AI/LLM API (e.g., OpenAI API)	Latest	Provides AI-based Q/A for document understanding inside the app.
	PDF Rendering Library (Flutter_pdfview / Syncfusion PDF Viewer)	Latest	Used to build the in-app PDF reader without third-party apps.

1.10 Project Deliverables

1. **Project Proposal Document:** A detailed outline of the Notes House project, including objectives, scope, problem statement, and proposed solution.
2. **Software Requirements Specification (SRS):** Complete documentation of system requirements, including functional, non-functional, user roles, and use cases.
3. **System Design Document:** Architectural diagrams, database schema, data flow diagrams, module design, and backend structure for the Notes House app.
4. **User Interface Prototypes:** Visual designs of app screens created using tools like Figma, showing student, teacher, and admin interfaces.
5. **Implementation (Source Code):** Full application code developed in Flutter (Dart), including frontend, backend integration, PDF viewer, AI chat, and secure download modules.
6. **Testing Report:** Testing plan, test cases, and results covering app functionality, UI/UX testing, performance, and security checks.
7. **User Manual:** A guide explaining how to use each feature of the app, including accessing notes, downloading files, taking quizzes, and using the AI chat system.
8. **Final Project Report:** Comprehensive documentation describing the entire project development, methodologies, results, challenges, and outcomes.
9. **Presentation Slides:** PowerPoint presentation summarizing the project for FYP evaluation, including visuals and key features.

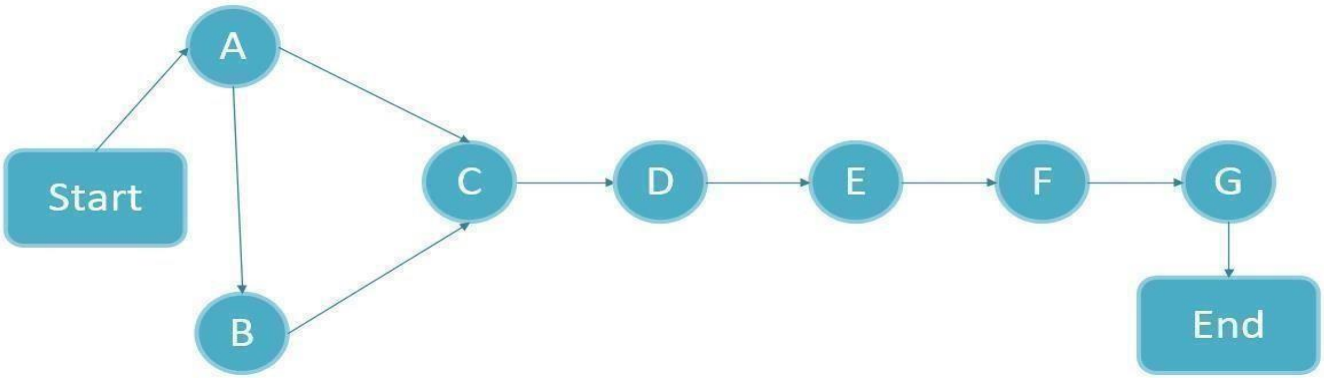
10. **Gantt Chart:** A detailed project timeline showing phases, milestones, tasks, and resource allocation throughout the development cycle.

1.11 Project Planning

1. **Phase 1 – Planning (Weeks 1–3):**
   Prepare project proposal, finalize idea, and complete SRS documentation.
2. **Phase 2 – Requirement Gathering (Weeks 4–7):**
   Collect system requirements, design UI/UX wireframes, and create the database schema.
3. **Phase 3 – Analysis (Weeks 8–10):**
   Analyze app modules such as notes upload, admin approval, PDF reader, quizzes, and AI chat.
4. **Phase 4 – Design (Weeks 11–15):**
   Design screen layouts, system architecture, backend structure, and app navigation.
5. **Phase 5 – Coding (Weeks 16–21):**
   Develop major modules including student access, teacher uploads, admin panel, PDF reader, secure downloads, quizzes, and AI chatbot.
6. **Phase 6 – Debugging & Testing (Weeks 22–23):**
   Fix bugs, perform unit and integration testing, and prepare the testing report.
7. **Phase 7 – Deployment & Presentation (Week 24):**
   Install the complete app, finalize documentation, and prepare presentation slides for the final defense.

1.11.1 Network Diagram

Network diagram of the activities is as under:



1.11.2 Estimate Activity Completion Time

The time required to complete each activity can be estimated using experience or by estimate of knowledge person.

Task ID	Predecessors	Duration(weeks)
A	**-**	**3 weeks**
B	**A**	**4 weeks**
C	**A, B**	**3 weeks**
D	**C**	**5 weeks**
E	**D**	**6 weeks**
F	**E**	**2 weeks**
G	**F**	**1 week**

1.11.3 Identify the Critical Path

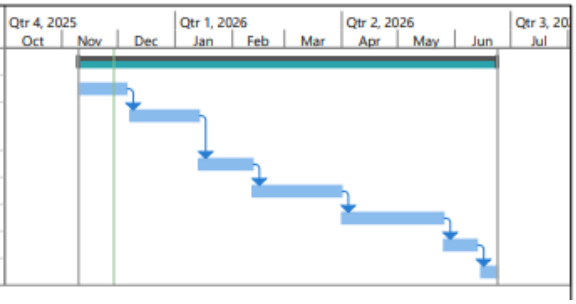
The critical path is the longest path through the network. The critical path for the above network diagram is: A B C D E F G H = 3+4+3+5+6+2+1 = 24

Task	Dependencies	Duration	ES	EF	LS	LF	TS	FS
A	-	3 weeks	0	3	0	3	0	0
B	A	4 weeks	3	7	3	7	0	0
C	A, B	3 weeks	7	10	7	10	0	0
D	C	5 weeks	10	15	10	15	0	0

E	D	6 weeks	15	21	15	21	0	0
F	E	2 weeks	21	23	21	23	0	0
G	F	1 week	23	24	23	24	0	0

1.11.4 Gantt Chart

ID	WBS	Task Name	Start	Finish	Duration
1	**1**	**Notes House**	**Mon 11/10/25**	**Tue 6/23/26**	**33.6 wks**
2	1	Planning	Mon 11/10/25	Fri 12/5/25	21 days
3	2	Requirement Gathering	Mon 12/8/25	Tue 1/13/26	28 days
4	3	Analysis	Wed 1/14/26	Wed 2/11/26	21 days
5	4	Design	Thu 2/12/26	Tue 3/31/26	35 days
6	5	Coding	Wed 4/1/26	Mon 5/25/26	42 days
7	6	Debugging	Tue 5/26/26	Fri 6/12/26	14 days
8	6	Deployment	Mon 6/15/26	Tue 6/23/26	7 days



1.11.5 Introduction to Team Members and Their Skill Set

1)	**Hasnat Ahmad**	**2022-gcg-123**
2)	**Abdul Hanan**	**2022-gcg-126**
3)	**Hafiz Abdul Rehman**	**2022-gcg-149**

Hasnat Ahmad:

Hasnat Ahmad is the group leader. He is responsible for the overall working of the Notes House project. His duties include managing the backend development, handling Firebase services, integrating APIs, managing teacher uploads, admin approval flow, secure file handling, and ensuring smooth communication between frontend and backend components.

Abdul Hanan:

Abdul Hanan is responsible for preparing the project documentation in a clear and standardized format. He also works on front-end UI/UX designing, screen layouts, graphic designing, and helping in structuring the app to make it simple and user-friendly for students and teachers.

Hafiz Abdul Rehman:

Hafiz Abdul Rehman handles the AI ChatBot module. He is responsible for implementing AI-based question answering, integrating the AI API, preparing the document-based retrieval system, and making sure the chatbot provides accurate results. He also assists in testing, debugging, and validating different parts of the application.

1.12 Summary

In this chapter, we explained the main idea and purpose of the Notes House project. We discussed the problems students and teachers face while searching for proper study material and how our app provides a complete and organized solution. The chapter also covered the system objectives, scope, limitations, constraints, and the tools and technologies that will be used to develop the application. We reviewed existing apps to identify gaps and highlighted how Notes House offers verified content, teacher uploads, admin approval, quizzes, protected downloads, and an AI chatbot for better learning. Additionally, we described the project deliverables and the detailed project plan, including timelines, phases, and task scheduling. These elements help set a clear direction for the successful development of the application, ensuring that the project stays aligned with its intended goals and provides meaningful benefits to university students across Pakistan.

Chapter 2

Requirement Analysis

2 Analysis

Provide an introduction to the chapter in a few lines. Write details in each of the sections provided ahead.

2.1 User classes and characteristics

User Class	Characteristics
Students	Students are the primary users of the app. They use Notes House to access notes, past papers, outlines, quizzes, download files securely, and use the AI chatbot for understanding topics. They need a simple interface to quickly find semester-wise study material.
Teachers	Teachers upload notes, outlines, and past papers after verification. They need an easy way to upload files, manage their content, and receive approval or rejection feedback from admins. A valid edu email is required for account creation.
Admins	Admins manage the entire system. They approve or reject notes uploaded by teachers, manage user accounts, review quizzes, and ensure data quality. They require full access to backend tools, logs, and dashboards.
AI ChatBot Users (Students/Teachers)	Any user who interacts with the AI chatbot to ask questions about study material. They need accurate and fast responses, especially when reading PDFs or solving questions.

2.2 Requirement Identifying Technique

For the **NotesHouse** project, the following requirement identification techniques are used:

2.2.1 Use Case Analysis:

1.1 This technique helps in identifying the major functions of the app based on interactions of Students, Teachers, and Admins.

1.2 Each use case represents a specific task such as accessing notes, uploading study material, approving content, taking quizzes, or interacting with the AI chatbot.

1.3 The main use cases include:

1.3.1 **UC1: Student Sign Up / Login**

1.3.2 **UC2: Teacher Sign Up (with edu email verification)**

1.3.3 **UC3: View Notes, Downloaded Notes and Quizzes**

1.3.4 **UC4: Upload Notes (Teacher)**

1.3.5 **UC5: Admin Approval / Rejection of Notes**

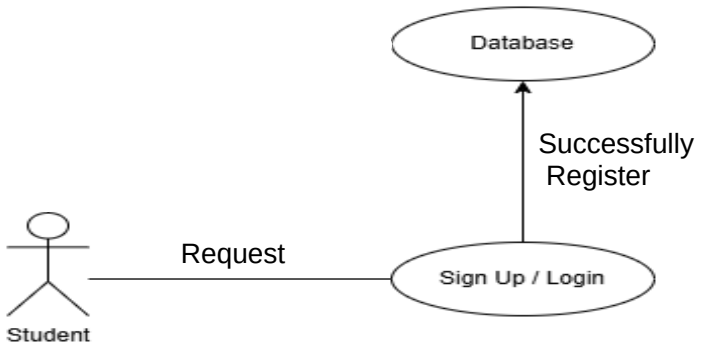
1.3.6 **UC6: Attempt Quiz and View Results**

1.3.7 **UC7: Use AI ChatBot for Study Help**

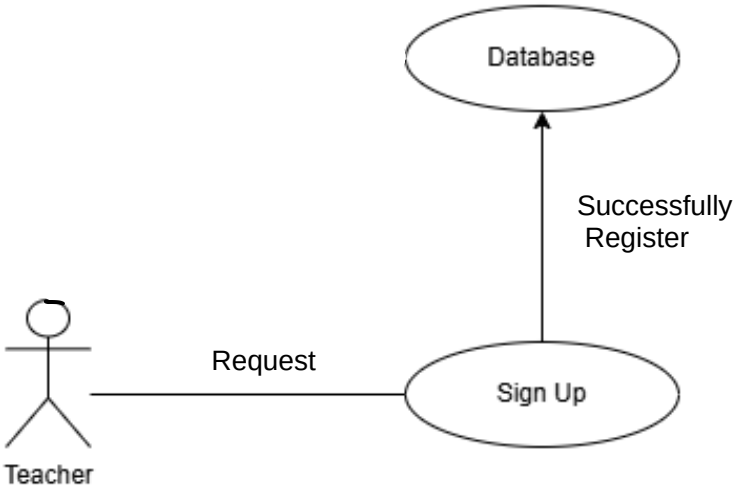
1.3.8 **UC8: Manage Profile & Settings**

1.4 A detailed use case diagram and fully dressed use case descriptions will be provided to explain each function clearly.

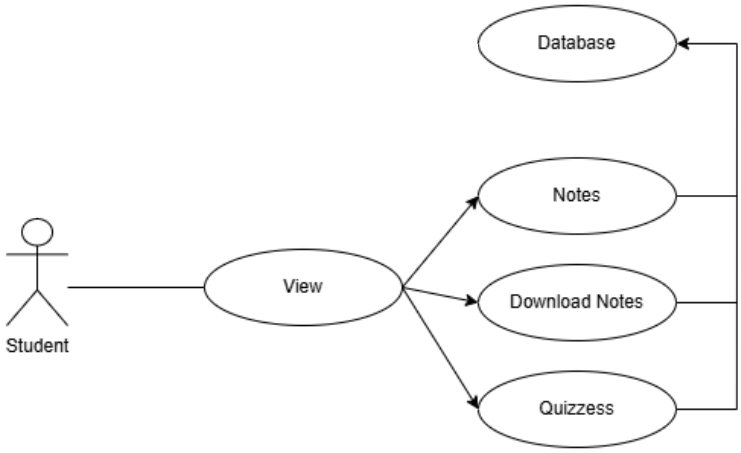
UseCase 01:



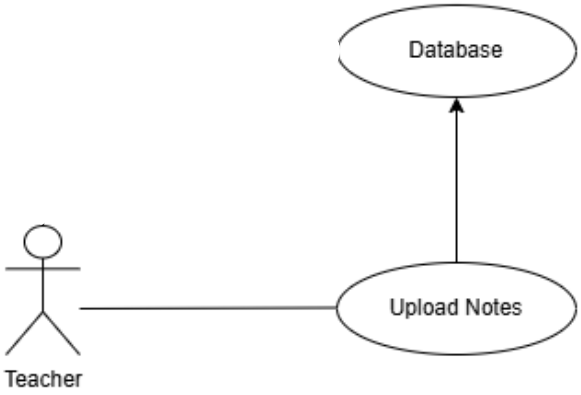
UseCase 02:



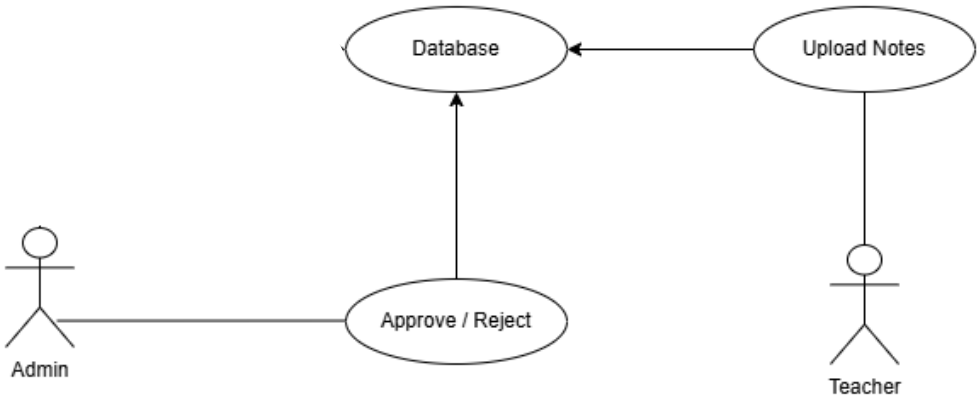
UseCase 03:



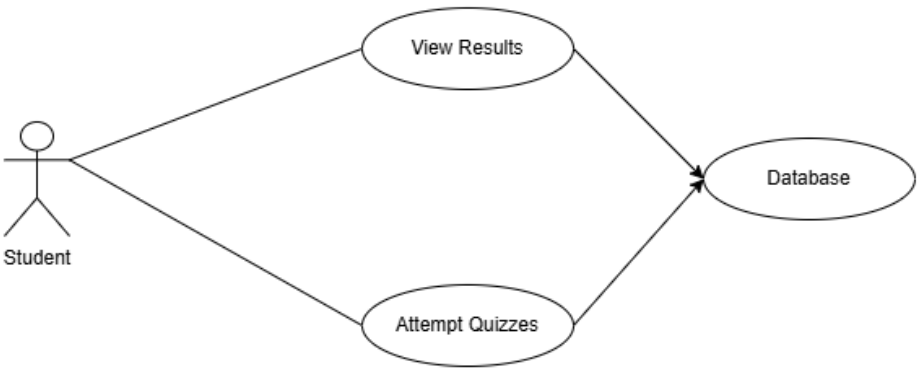
Use Case 04:



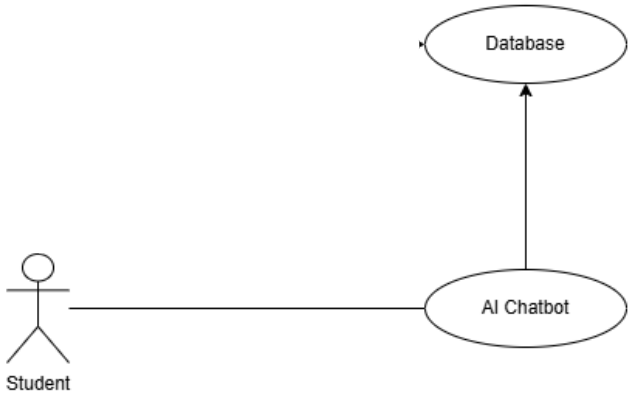
Use Case 05:



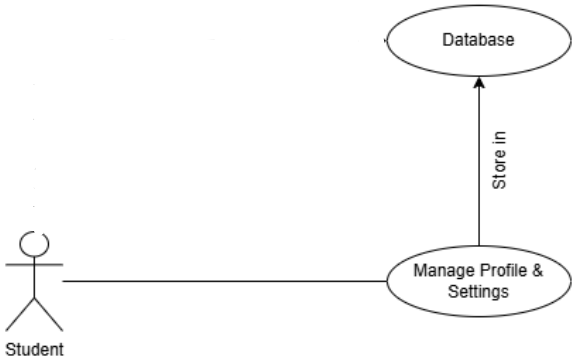
Use Case 06:



Use Case 07:



Use Case 08:



2.3 Functional Requirements

The functional requirements of the **Notes House** system are grouped according to the main features of the application. Each feature is explained along with its detailed functional requirements.

1. User Registration & Authentication

1. The system shall allow students and teachers to create accounts using email and password.
2. The system shall verify teacher identity through a valid **edu email**.
3. The system shall authenticate users during login using stored credentials.
4. The system shall allow users to recover their account using a "Forgot Password" option.
5. The system shall maintain secure sessions for logged-in users.

2. Study Material Access (Notes, Past Papers, Outlines)

1. The system shall allow students to select their university, department, and semester to view available study material.
2. The system shall display notes, past papers, and outlines in organized categories.
3. The system shall allow students to view PDFs inside the app without third-party applications.
4. The system shall allow students to download files **only inside the app** to prevent external sharing.
5. The system shall provide search and filter options for quick access to study material.

3. Teacher Upload System

1. The system shall allow teachers to upload notes, past papers, outlines, and other study files.
2. The system shall save uploaded files in pending status until reviewed by the admin.
3. The system shall notify teachers whether their uploaded content is approved or rejected.
4. The system shall allow teachers to update or delete uploaded files before approval.

4. Admin Panel & Verification System

1. The system shall allow admins to review all uploaded study materials.
2. The system shall provide options to approve or reject notes based on quality and relevance.
3. The system shall notify teachers instantly after approval or rejection.
4. The system shall allow admins to manage users, including Teacher approval or deleting accounts.
5. The system shall allow admins to monitor system activity logs and reports.

5. Quiz Module

1. The system shall allow students to attempt quizzes related to selected subjects.
2. The system shall calculate the quiz score automatically after submission.
3. The system shall display quiz results and correct answers to students.
4. The system shall store quiz history for performance comparison.

6. Quiz Analysis & Performance Reports

1. The system shall generate graphical reports (charts/graphs) based on quiz history.
2. The system shall store past quiz results for future analysis.

7. AI ChatBot Assistance

1. The system shall allow users to open the AI chatbot screen.
2. The system shall allow users to open notes inside the chatbot for asking questions.
3. The system shall process user questions and provide relevant answers based on study material.
4. The system shall allow users to open the AI chatbot screen anytime from the app.
5. The system shall allow users to ask general study-related questions even without opening a PDF.
6. The system shall maintain the chat history on the screen until the user clears it manually.

8. In-App PDF Viewer

1. The system shall allow users to read notes and papers inside the app.
2. The system shall allow zoom-in and zoom-out functionality.
3. The system shall allow page navigation within the PDF.
4. The system shall integrate the “Ask AI” button to switch from reading to chatbot mode.

9. Downloads Manager

1. The system shall allow students to download notes inside the app.
2. The system shall store all downloaded files in an internal app folder.
3. The system shall allow students to access downloaded files offline.
4. The system shall prevent users from sharing files outside the app.

10. Profile & Settings Management

1. The system shall allow users to update their profile information.
2. The system shall allow users to change their password.

3. The system shall allow users to log out from their account.

2.3.1 Functional Requirement X

Itemize the specific functional requirements associated with each feature. These are the software capabilities that must be implemented for the user to carry out the feature's services or to perform a use case. Describe how the product should respond to anticipated error conditions and to invalid inputs and actions. Uniquely label each functional requirement, as described earlier. You can create multiple attributes for each functional requirement, such as rationale, source, dependencies, etc. The following template is required to write functional requirements.

Table 1- 1Description of FR-1

Identifier	FR-1
Title	**User Registration (Student & Teacher)**
Requirement	The system shall allow users (students and teachers) to create an account using email and password. Teachers must register with a valid edu email.
Source	Student, Teacher
Rationale	To ensure that only verified users can access study material or upload content.
Business Rule (if required)	Teacher registration must be validated using edu domain (e.g *.edu.pk*)
Dependencies	Database module, Authentication module
Priority	High
Identifier	FR2
Title	**User Login**
Requirement	The system shall authenticate registered users and allow them to log in using their email and password.
Source	System

Rationale	To provide secure access to user-specific data and prevent unauthorized use.
Business-Rule (if required)	Invalid credentials should show proper error messages
Dependencies	FR-1 (Registration), Database
Priority	High
Identifier	**FR3**
Title	**View Study Material**
Requirement	The system shall allow students to view notes, past papers, and course outlines according to their selected semester.
Source	Student
Rationale	Helps students easily access organized study content.
Business Rule (if required)	Material must be categorized by subject and semester.
Dependencies	Database, UI module
Priority	High
Identifier	**FR4**
Title	**In-App PDF Viewer**
Rationale	Ensures security and licensing rights of notes
Business Rule (if required)	File must be saved in internal app storage only.
Dependencies	FR-4, Download Manager Module

Priority	High
Identifier	**FR5**
Title	**Protected Download System**
Requirement	The system shall allow users to download study material only inside the app, without exporting it externally.
Source	Student
Rationale	Ensures security and licensing rights of notes.
Business Rule (if required)	File must be saved in internal app storage only.
Dependencies	FR-4, Download Manager Module
Priority	High
Identifier	**FR-6**
Title	**Upload Notes (Teacher)**
Requirement	The system shall allow teachers to upload notes, outlines, or past papers and send them for admin approval.
Source	Teacher
Rationale	Allows teachers to contribute verified academic content.
Business Rule (if required)	Only PDF or image formats allowed.
Dependencies	FR-1, Database
Priority	High

Identifier	**FR-7**
Title	**Admin Approval / Rejection**
Requirement	The system shall allow admins to approve or reject uploaded notes based on quality and relevance.
Source	Admin
Rationale	Ensures study material is validated before students see it.
Business Rule (if required)	Admin must add a reason for rejection.
Dependencies	FR-6
Priority	High
Identifier	**FR-8**
Title	**Quiz Attempt**
Requirement	The system shall allow students to attempt quizzes related to each subject.
Source	Student
Rationale	Helps students test their preparation.
Business Rule (if required)	Quiz scoring must be calculated automatically.
Dependencies	Question Bank Module
Priority	Medium
Identifier	**FR-9**

Title	**Quiz Result & Analysis**
Requirement	The system shall store quiz attempts and display performance graphs.
Source	Student
Rationale	Helps students track improvement.
Business Rule (if required)	Graphs must be generated from stored quiz history.
Dependencies	FR-8
Priority	Medium
Identifier	**FR-10**
Title	**AI ChatBot Assistance**
Requirement	The system shall allow users to interact with the AI in two modes: general chat mode (without notes) and context-based mode (with notes opened in the internal PDF viewer).
Source	Student
Rationale	Helps students understand concepts and ask questions about notes.
Business Rule (if required)	PDF must be internal; external uploads not allowed.
Dependencies	PDF Viewer, AI Module
Priority	High
Identifier	**FR-11**
Title	**Manage User Profile**

Requirement	The system shall allow users to update their profile information, including name, password, and notification settings.
Source	Student, Teacher, Admin
Rationale	Allows users to keep their information updated and manage personalization.
Business Rule (if required)	Password changes must require re-authentication.
Dependencies	Authentication Module
Priority	Medium
Identifier	**FR-12**
Title	**User Logout**
Requirement	The system shall allow users to securely log out of the application.
Source	System
Rationale	Ensures account privacy and prevents unauthorized access.
Business Rule (if required)	User session must be terminated upon logout.
Dependencies	FR-1, FR-2
Priority	High
Identifier	**FR-13**
Title	**Search and Filter Notes**
Requirement	The system shall allow users to search and filter study material by subject, semester, file type, or keywords.

Source	Student
Rationale	Helps users find notes quickly and improve usability.
Business Rule (if required)	Search results must match keywords from titles or tags.
Dependencies	Database Module
Priority	Medium
Identifier	**FR-14**
Title	**User Account Management (Admin)**
Requirement	The system shall allow admins to manage user accounts, including activating or deactivating users.
Source	Admin
Rationale	Ensures better control over student and teacher accounts.
Business Rule (if required)	Only Admin can deactivate an account.
Dependencies	Authentication Module
Priority	High
Identifier	**FR-15**
Title	**Push Notifications**
Requirement	The system shall send notifications to students and teachers regarding approvals, quiz results, and important updates.
Source	System

Rationale	Keeps users informed about important events.
Business Rule (if required)	Notifications must only be sent to active users.
Dependencies	Database, Admin Panel
Priority	Medium
Identifier	**FR-16**
Title	**In-App Download Manager**
Requirement	The system shall store all downloaded files in an internal app folder and allow users to access them offline.
Source	Student
Rationale	Provides secure offline access without external sharing.
Business Rule (if required)	File export must be disabled.
Dependencies	FR-4, FR-5
Priority	High
Identifier	**FR-17**
Title	**File Access Control**
Requirement	The system shall prevent files from being copied, shared, or taken outside the app.
Source	System
Rationale	Protects the confidentiality and ownership of study material.

Business Rule (if required)	External storage permission must be blocked.
Dependencies	Download Manager, PDF Viewer
Priority	High
Identifier	**-**
Title	**Maintain AI Chat History**
Requirement	The system shall display the ongoing conversation history until the user manually clears it.
Source	Student
Rationale	Allows users to follow their previous questions easily.
Business Rule (if required)	History must clear when user remove manually.
Dependencies	AI Chat Module
Priority	Medium
Identifier	**FR-19**
Title	**Switch Between PDF and Chatbot**
Requirement	The system shall allow users to switch smoothly between the PDF viewer and the AI chatbot without losing context.
Source	Student
Rationale	Improves user experience by allowing back-and-forth navigation.
Business Rule (if required)	PDF must remain open in the background.

Dependencies	PDF Viewer, AI Module
Priority	Medium
Identifier	**FR-20**
Title	**Activity Log for Admin**
Requirement	The system shall maintain logs of uploads, approvals, rejections, login activities, and quiz usage for admin analysis.
Source	Admin
Rationale	Ensures transparency and helps monitor system usage.
Business Rule (if required)	Logs must be timestamped.
Dependencies	Database, Admin Panel
Priority	Medium

2.4 Non-Functional Requirements

This section defines the quality attributes of the **Notes House** application. These requirements describe how well the system should perform rather than what the system should do.The NFRs are grouped into four categories:

- Reliability
- Usability
- Performance
- Security

2.4.1 Reliability

NFR1: Mean Time Between Failures (MTBF)

1.1 The system shall maintain a minimum MTBF of **200 hours** for critical functions such as notes viewing, quiz attempts, and file uploading.

1.2 The system shall operate smoothly under normal conditions without frequent crashes or app freezes.

NFR2: Failure Recovery

2.1 In case of app crash, the application shall automatically recover the previous state (e.g., last opened PDF, ongoing quiz) within **2 minutes**.

2.2 All unsaved data (e.g., quiz progress) shall be recovered using cached storage.

NFR3: Error Detection and Correction

3.1 The system shall detect invalid inputs (e.g., unsupported file formats) and prompt users with clear instructions.
3.2 System logs shall record all errors for developers to analyze and fix issues.

2.4.2 Usability

NFR4: Ease of Learning

4.1 New users shall be able to understand and use the app's basic features (view notes, attempt quiz, use AI chatbot) within **10 minutes** of first use.

NFR5: Ease of Use

5.1 The user interface shall be simple, clean, and easy to navigate with clear labels for all features.
5.2 Large buttons and readable fonts shall be used for better accessibility.
5.3 The app shall provide tooltips or short guides for first-time users.

NFR6: Accessibility

6.1 The system shall be usable by users of different ages and technical skills, including non-technical students.
6.2 The app shall support dark mode for better readability and reduced eye strain.

2.4.3 Performance

NFR7: Response Time

7.1 The system shall load notes and past papers within **3 seconds**.
7.2 Quiz results shall be displayed within **2 seconds** after completion.
7.3 AI chatbot responses shall be generated within **2–4 seconds**, depending on question complexity.

NFR8: File Upload and Download Performance

8.1 Teacher uploads (PDF/DOC) up to **25 MB** shall complete within **8 seconds** under normal internet conditions.
8.2 Student downloads within the app storage shall complete within **5 seconds**.

NFR9: Data Processing Time

9.1 Quiz analytics (charts, results, performance graphs) shall be generated within **4 seconds**.

NFR10: Scalability

10.1 The system shall support up to **10,000 users** concurrently without affecting performance.
10.2 The database shall scale automatically when the number of uploaded notes increases.

2.4.4 Security

NFR11: Data Protection

11.1 All notes, past papers, user data, and quiz scores shall be stored securely using **AES-256** encryption.
11.2 Sensitive information such as teacher email (edu email) and admin credentials shall be protected.

NFR12: Authentication & Authorization

12.1 Only registered students and verified teachers shall access the app.
12.2 The system shall restrict access based on user roles (Student / Teacher / Admin).

NFR13: Data Breach Detection

13.1 The system shall monitor suspicious login attempts and notify the admin within **5 minutes**.
13.2 Multiple failed login attempts shall lock the account temporarily.

NFR14: Backup & Data Recovery

14.1 Daily database backups shall be taken automatically.
14.2 Backup data shall be securely stored for recovery in case of system failure.

2.5 External Interface Requirements

This section provides information to ensure that the system will communicate properly with users and with external hardware or software elements. A complex system with multiple subcomponents should create a separate interface specification or system architecture specification. The interface documentation could incorporate material from other documents by reference. For instance, it could point to a hardware device manual that lists the error codes that the device could send to the software.

2.5.1 User Interface Requirements

The user interface of the **Notes House** app shall follow the guidelines below:

GUI Standards

1. The interface shall comply with **Material Design** guidelines to maintain consistency across screens.
2. All icons, fonts, and buttons shall follow Android design standards for clarity and usability.
3. The UI shall maintain a simple layout to ensure easy navigation for students and teachers.

Standard Buttons and Navigation Links

1. Each major screen shall include **Home**, **Back**, and **Profile** buttons for smooth navigation.
2. The app shall provide a **Search** button on notes and past papers screens to easily locate study material.
3. A **Help/Support** button shall be available for contacting support or accessing FAQs.

Message Display Conventions

1. Error messages shall appear in **red** at the top of the screen with a clear description.
2. Confirmation or success messages shall be displayed in **green**, along with a timestamp.
3. Warning messages (e.g., incomplete upload, unstable network) shall be shown in **yellow**.

Screen Layout & Resolution Constraints

1. The application shall support screen resolutions ranging from **720×1280** to **1080×2400** pixels.
2. All UI elements shall automatically adjust using Flutter's responsive layout capabilities.

Accessibility Accommodations

1. The system shall support screen readers for visually impaired students.
2. The interface shall offer sufficient color contrast for easy readability.
3. Dark mode shall be supported to reduce eye strain for prolonged study use.

2.5.2 Software Interfaces

The system shall interface with the following external software components:

Database System

1. The application shall connect to **Firebase Firestore** for storing notes, past papers, quizzes, chat history, and user data.

Authentication System

1. The system shall use **Firebase Authentication** for student, teacher, and admin login.
2. Teachers must verify their identity using a valid **.edu** email.

Operating System

1. The application shall be compatible with **Android OS version 8.0 and above**.

AI Chatbot API

1. The system shall integrate with an external AI API (such as OpenAI / Gemini) to generate chatbot responses.

2.5.3 Hardware Interfaces

The system shall interface with the following hardware components:

Mobile Device Sensors

1. The application shall access the device's **notification module** to display reminders and alerts.
2. The system shall access device storage for saving offline study material inside app-specific storage.

Network Connectivity

1. The application shall require **Wi-Fi or mobile data** for synchronizing notes, chatting with AI, and uploading files.
2. In offline mode, only previously downloaded notes and quizzes shall be accessible.

2.5.4 Communications Interfaces

Notification Service

1. The system shall use **Firebase Cloud Messaging (FCM)** to deliver notifications for:
    - New notes uploaded
    - Quiz results
    - Teacher announcements

Emergency Communication (if applicable)

1. If the user enables alerts, the application shall transmit notifications to designated contacts (admin or teacher).

Server Communication

1. All communication with Firebase and AI APIs shall use **HTTPS** for secure data transmission.
2. The system shall sync updates (uploads, downloads, quiz analytics) whenever internet connectivity is available.

2.6 Summary

Requirements Identification:

1. This process involved communicating with stakeholders, understanding their challenges in accessing notes, quizzes, and study material, and prioritizing their needs.
   It helped define a clear system scope and ensured that the proposed solution focuses on the real problems faced by university students and teachers.

Use Case Analysis:

1. The use case technique was used to describe how different users (students, teachers, admins) interact with the system.
   It clarified user goals, system behavior, and the sequence of actions required to perform tasks.
   This approach helped specify functional requirements from the user's point of view.

Importance of Requirements Engineering:

1. Requirements engineering ensures that the final application meets actual user needs and avoids misunderstandings during development.
2. It guides the development process, reduces risks, and minimizes costly changes at later stages.
3. It provides a structured foundation for design, implementation, and testing, increasing the chances of project success.
4. Clearly documented requirements help the team stay aligned and maintain consistent progress throughout the project.

Chapter 3

Design and Architecture

3. System Design

Notes House

1. Presentation Layer (Frontend / User Interface)

Purpose: Interface where users (Students, Teachers, and Admins) interact with the system.

Technologies: Flutter (for development), Firebase (to store and retrieve data), Node.js & Express (On localhost)

Components:

1. **Login/Registration Interface** – Secure login/signup for Students, Teachers (edu email), and Admins
2. **Dashboard Screen** – Shows semester options, notes, quizzes, downloads, and AI chatbot
3. **Study Material Viewer** – Displays notes, past papers, and outlines
4. **In-App PDF Reader** – Allows reading PDFs with zoom, navigation, and "Ask AI" option
5. **Quiz Interface** – Enables students to attempt quizzes and view results
6. **Teacher Upload Panel** – Teachers upload study material for admin approval
7. **Admin Review Panel** – Admins approve or reject uploaded files

2. Application Layer (Logic & Controller Layer)

Purpose: Handles business logic and manages interaction between frontend and backend.

Technologies: Dart (Flutter logic), Firebase Services

Key Controllers/Services:

- **Authentication Service** – Manages login, signup, and role-based access
- **Study Material Controller** – Loads and filters notes by semester
- **PDF Controller** – Handles PDF viewing and text extraction for AI
- **AI Chat Controller** – Processes user questions and communicates with AI API
- **Quiz Manager** – Handles quiz loading, evaluation, and score calculation
- **Upload Manager** – Processes teacher uploads and admin approvals

3. Data Layer (Backend / API & Database)

Purpose: Stores, retrieves, and processes all core system data.

Technologies: Firebase Firestore & Firebase Storage

Endpoints / Operations:

• **POST /upload** – Teacher uploads study material
• **GET /materials** – Retrieve approved study files
• **POST /quiz-submit** – Save quiz results
• **GET /quiz** – Retrieve quiz questions
• **POST /ai-query** – Send question to AI service
• **GET /downloads** – Retrieve user download history

4. Data Storage Layer (Database Design)

Database: Firebase Firestore & Firebase Storage

Collections / Tables:

• **Users** – Stores student, teacher, and admin information
• **StudyMaterials** – Stores notes, past papers, outlines, and metadata
• **UploadRequests** – Tracks teacher uploads with approval status
• **Quizzes** – Stores quiz questions and answers
• **QuizResults** – Logs student scores and performance history
• **AIChatSessions** – Saves chatbot questions and responses
• **Downloads** – Tracks in-app downloaded files

5. Notification & Scheduling Layer

Purpose: Handles real-time alerts and scheduled updates

Technologies: Firebase Cloud Messaging (FCM) – Used for push notifications

6. Admin Dashboard

Features:

• View system analytics
• Monitor uploads and approvals
• Manage users and study materials

Data Flow Overview

1. Student logs in → Requests study material → Firebase → Database
2. System retrieves approved material → Displays in app
3. Student opens PDF → Clicks “Ask AI” → Sends query to AI service
4. Teacher uploads file → Admin reviews → Status updated in database
5. System sends notifications for approvals, quiz results, or updates

3.1 Design Considerations

Assumptions and their descriptions:

Assumption / Dependency	Description
Reliable Internet Connection	The app assumes users have stable internet to access study materials, AI chatbot, and database updates.
Firebase Service Availability	The system depends on Firebase services for authentication, database access, and file storage.
AI Service Availability	The AI chatbot requires an active API connection to generate responses.
Edu Email Verification	Teachers must use a valid educational email for account verification.
Notification Service Availability	The system relies on Firebase Cloud Messaging for sending push notifications.
Device Storage Availability	Sufficient internal storage is required for downloading study materials.
Time Synchronization	Device time should be accurate for quiz attempts, uploads, and activity logs.
Admin Verification Process	Uploaded materials must be reviewed and approved by admin before becoming visible to students.
User Permissions Granted	Notification and storage permissions are required for proper app functionality.

Limitations:

Limitations	Description
No Real-Time Content Verification	Study materials are not automatically verified; admin approval is required before visibility.
Offline Functionality Limited	Most features such as AI chatbot and quiz attempts require an internet connection.
AI Response Accuracy	AI answers depend on available data and may not always be perfectly accurate.
Dependency on Mobile Device Features	Notifications and downloads depend on device permissions and storage availability.
Scalability for Large User Base	Performance may slow if user traffic increases without database optimization.
Limited Cross-Platform Support	Initial release may focus on mobile platform only.

Limitations (Risk Analysis):

Risk	Impact	Mitigation Strategy
Upload of Incorrect or Irrelevant Study Material	Students may access wrong or low-quality content	Admin approval process before publishing material
AI Response Inaccuracy	Students may receive incomplete or incorrect answers	Limit AI responses to uploaded content and allow follow-up clarification
Notification Failure	Users may miss updates about approvals or quizzes	Use Firebase Cloud Messaging with retry mechanism and logging
Unauthorized Access to User Data	Privacy and data security risk	Implement role-based access control and Firebase authentication
Server or Firebase Downtime	Temporary unavailability of materials or services	Use cloud-based scalable infrastructure and backup mechanisms
Excessive Storage Usage	App performance may slow due to large file downloads	Restrict file sharing outside app and manage internal storage

3.2 Design Models

Below is a detailed explanation and description of the models used to represent the **Notes House** system using an Object-Oriented Development Approach, along with visual diagram suggestions for:

• Class Diagram
• Interaction Diagram (Sequence Diagram)
• State Transition Diagram

Each model is described first, followed by diagrams that can be included in the project documentation.

1. Class Diagram

Description:

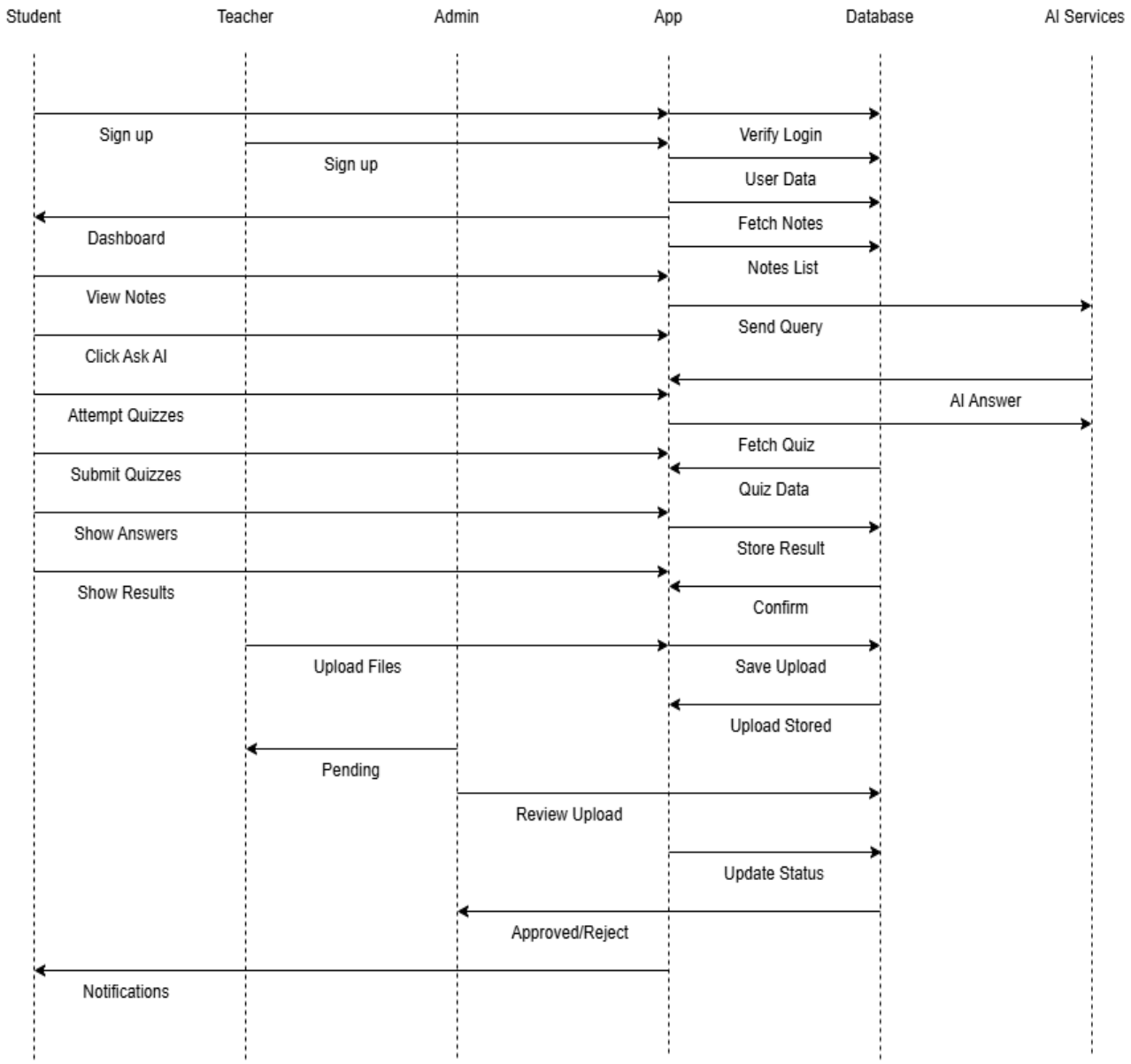
A class diagram illustrates the static structure of the system. It shows the main classes, their attributes, methods, and the relationships among them.

Main Classes:

User
Study Material
Upload Request
Quiz
Quiz Result
AIChat Session
Download Record

Relationships:

A User (Teacher) can upload multiple Study Materials through Upload Request.
A User (Admin) can approve or reject Upload Requests.
A User (Student) can attempt multiple Quizzes and generate multiple Quiz Results.
A Study Material can be downloaded multiple times by different Users.
An AIChat Session is linked to a User and stores question-answer interactions.



2. Interaction Diagram (Sequence Diagram)

Description:

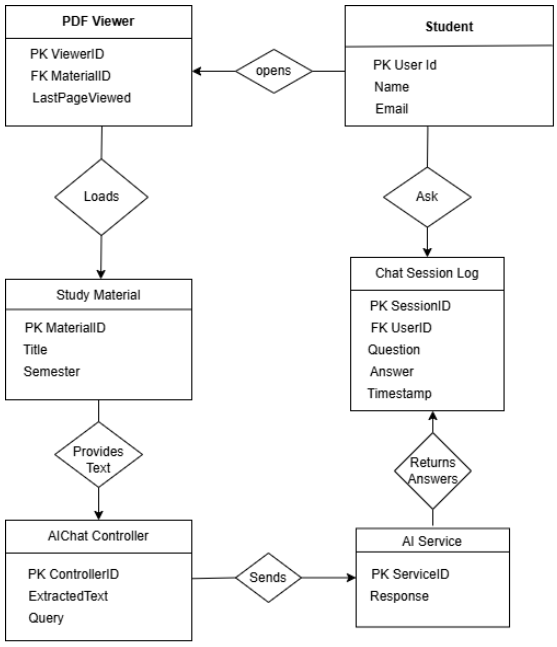
This diagram models the sequence of messages exchanged between objects during a specific use case. Here, we'll use the **"Student opens a PDF and asks a question using the AI chatbot"** scenario.

Objects in Sequence:

Student UI
PDFReader
AIChatController
AIService
Database

Flow:

• Student opens a study material PDF.
• PDFReader loads the file from the database.
• Student enters a question using the "Ask AI" option.
• AIChatController extracts the relevant text from the PDF.
• The question and extracted text are sent to the AIService.
• The AIService processes the query and returns a response.
• The response is displayed to the student.
• The chat interaction is saved in the database for history.



3. State Transition Diagram

Description:

This diagram shows the states and transitions of an object that reacts to user actions. In this case, we model the PDF Viewer with AI Interaction in the Notes House system.

States of PDF Viewer:

Closed (Viewer Off) → PDF viewer is not active.

Loading (Fetching From DB) → Selected PDF is being loaded from the database.

Open (PDF Visible) → PDF is successfully displayed to the student.

Navigating (Zoom / Scroll) → Student is reading the PDF using zoom and scroll.

Opened (Return to Read) → Viewer is active but user is not interacting.

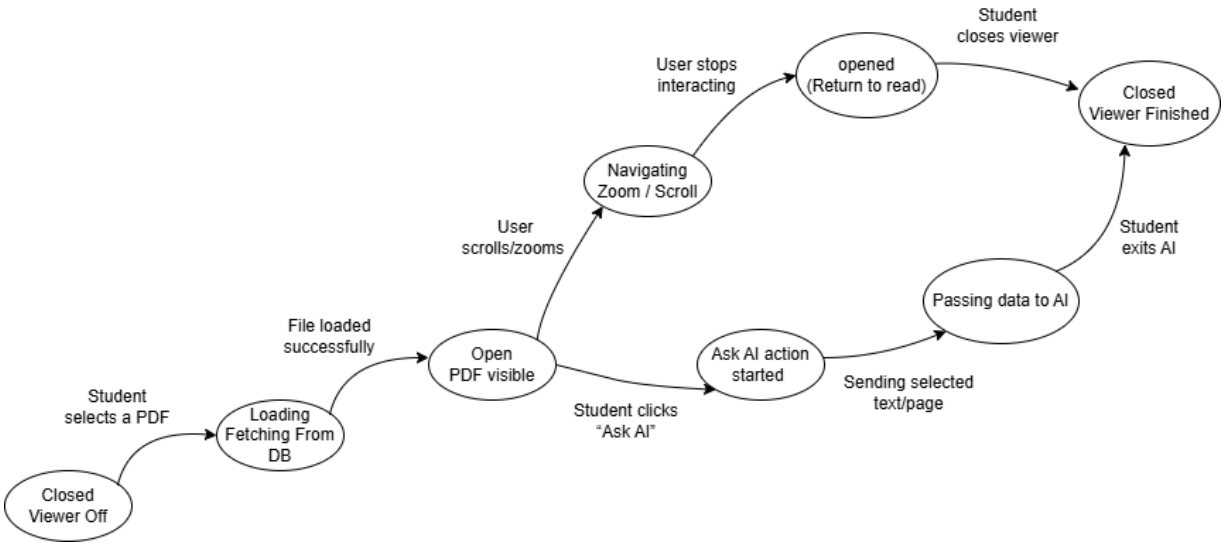
Ask AI Action Started → Student clicks the "Ask AI" option**.**

Passing Data to AI → Selected text/page is sent to AI for processing**.**

Closed (Viewer Finished) → Viewer is closed after reading or AI interaction

Events:

- Student selects a PDF
- File loads successfully
- User scrolls or zooms
- User stops interaction
- Student clicks "Ask AI"
- System sends selected content to AI
- Student exits AI
- Student closes the viewer



3.3 Architectural Design

This section explains how the main components of the **Notes House** system work together. The architecture shows the interaction between the user interface, application logic, and Firebase backend.

The system is based on a **Layered/MVC architecture**, which is organized into:

• Presentation Layer (View)

Login screens, dashboards, study material viewer, PDF reader, AI chat, upload and admin panels.

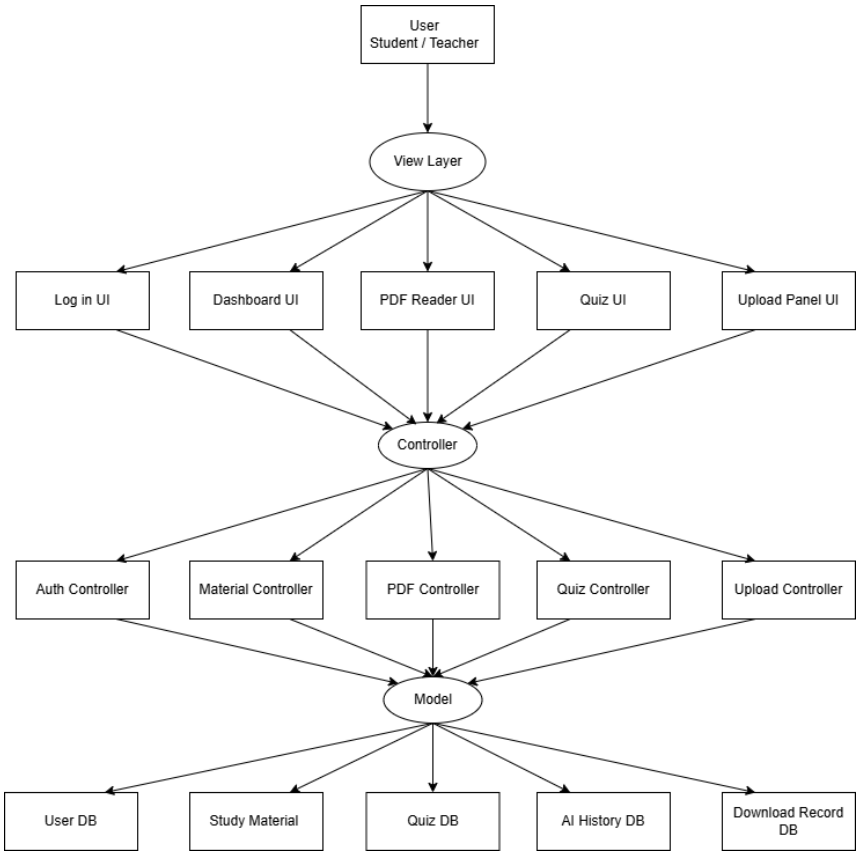
• Application Layer (Controller)

Authentication service, study material controller, PDF controller, AI chat controller, quiz manager, upload manager, and admin review controller.

• Data Layer (Model)

Firebase Authentication, Firestore Database, Firebase Storage, and in-app file storage.

UML Class and Component Diagrams represent how these modules are connected and how data flows between them to provide all system features like notes access, quizzes, AI chat, uploads, and admin approvals.



3.4 Data Design

The information domain of the Notes House system focuses on managing study materials, teacher uploads, quizzes, AI chat history, and student downloads. These data items are organized into structured entities stored in Firebase Firestore and Storage to ensure consistency, scalability, and quick access.

1. Major Entities and Their Storage

- **User**

  Stores basic details of Students, Teachers, and Admins.
  Fields: user_id, name, email, role, profile_info

- **Study Material**

  Stores notes, past papers, outlines, and related metadata.
  Fields: material_id, title, type, semester, file_url, uploaded_by

- **Upload Request**

  Represents files submitted by teachers for admin approval.
  Fields: request_id, material_id, teacher_id, timestamp, status

- **Quiz**

  Contains questions and answers for each subject or semester.
  Fields: quiz_id, subject, semester, questions, correct_answers

- **Quiz Results**

  Stores performance records of students for analysis.
  Fields: result_id, student_id, quiz_id, score, timestamp

- **AI Chat Sessions**

  Logs all interactions between the user and the AI chatbot.
  Fields: session_id, user_id, question, answer, timestamp

- **Downloads**

  Tracks files downloaded within the app for offline use.
  Fields: download_id, user_id, material_id, timestamp

2. Data Processing and Logic

- **Study Material Access:**

  When a student selects a semester or subject, the system fetches related notes, past papers, and outlines from the database and displays them.

• **AI Chat Processing:**

When a student clicks “Ask AI,” the system extracts text from the opened PDF, sends it with the question to the AI service, and returns the generated answer.

• **Teacher Upload Handling:**

Teachers upload study files which are stored as pending. The system forwards these uploads to the admin for review and updates the status after approval or rejection.

• **Admin Review Logic:**

Admins verify uploaded files, update their approval status, and make approved material visible to all students.

• **Quiz Processing:**

The system loads quiz questions, checks answers submitted by students, computes the score, and stores results for analysis.

• **Download Management:**

When a student downloads a file, the system saves it in internal storage and logs the download record for offline access.

3. Data Structure Organization

• Database Design:

- Collections are structured to keep data organized and avoid redundancy.
- References (foreign keys) link users, study material, quizzes, uploads, and chat history.

• Security and Privacy:

- User data is protected using Firebase Authentication and secure access rules.
- Study files and chat history are access-controlled to prevent unauthorized visibility.

• Scalability:

- Firestore indexing on fields like user_id, semester, and timestamps ensures fast queries.
- Firebase’s cloud-based storage and auto-scaling support growing users and materials

3.4.1 Data Dictionary

1. Alphabetical List of System Entities and Data

Term	Description
aiAnswer	String: AI chatbot response text.
aiQuestion	String: Question asked by the student.
chatSession	Object: Stores question, answer, and timestamp.
downloadID	String/UUID: Unique ID for downloaded material.
fileType	String: Type of study material (notes, outline, past paper).
materialID	String/UUID: Unique ID for each study file.
materialTitle	String: Name/title of the study material.
pdfPage	Integer: Current page viewed in the PDF reader.
quizID	String/UUID: Unique ID for each quiz.
quizScore	Integer: Marks obtained by student.
quizTimestamp	DateTime: When the quiz was attempted.
requestStatus	String: Upload status (Pending, Approved, Rejected).
requestID	String/UUID: Unique ID for teacher upload request.
role	String: User role (Student, Teacher, Admin).
studentID	String/UUID: Unique ID for students.
subjectName	String: Name of subject for quiz or material.
teacherID	String/UUID: Unique ID for teachers.
uploadDetails	Object: Contains metadata about teacher-uploaded files.
uploadTimestamp	DateTime: Time when teacher submitted a file.
userEmail	String: Email used for login.
userName	String: Display name of the user

2. Object-Oriented Approach: Objects, Attributes, Methods & Parameters

Class: StudyMaterial

Attributes:
• materialID: String
• title: String
• type: String
• semester: String
• fileURL: String
• uploadedBy: Teacher

Methods:
• getDetails() – Returns material info.
• isApproved() – Checks if file is approved.

Class: UploadRequest

Attributes:
• requestID: String
• material: StudyMaterial
• teacher: Teacher
• status: String (Pending, Approved, Rejected)
• timestamp: DateTime

Methods:
• approve(adminID: String) – Marks request as approved.
• reject(adminID: String) – Marks request as rejected.

Class: User

Attributes:
• userID: String
• name: String
• email: String
• role: String

Methods:
• getProfile() – Returns user data.
• updateProfile(data: Object) – Updates user info.

Class: Quiz

Attributes:
• quizID: String
• questions: List
• subject: String
• semester: String

Methods:
• evaluate(answers: List) – Computes score.
• getQuestions() – Returns quiz items.

Class: QuizResult

Attributes:
• resultID: String
• quiz: Quiz
• student: User
• score: Integer
• timestamp: DateTime

Methods:
• getResult() – Returns score details.

Class: AIChatSession

Attributes:
• sessionID: String
• user: User
• question: String
• answer: String
• timestamp: DateTime

Methods:
• saveSession() – Stores chat in database.
• isFollowUp() – Checks if user asked a related question.

Class: PDFReader

Attributes:
• material: StudyMaterial
• currentPage: Integer

Methods:
• loadPDF() – Loads the selected file.
• extractText(page: Integer) – Returns text for AI.

Class: DownloadManager

Methods:
• saveDownload(userID: String, materialID: String) – Stores offline record.
• getDownloads(userID: String) – Retrieves user downloads.

3.5 User Interface Design

The Notes House application is designed with a simple and user-friendly interface to help students, teachers, and admins access study materials, attempt quizzes, upload notes, and interact with the AI chatbot smoothly. From the user's viewpoint, the app provides a clean workflow that makes it easy to browse subjects, open PDFs, ask questions through AI, attempt quizzes, and manage uploads or approvals. The interface ensures that all users can navigate the system with ease and complete their tasks quickly.

User Interaction and Features

Study Material Access:

Students can browse notes, past papers, and outlines using clear subject and semester filters. Files can be opened instantly using the in-app PDF viewer for smooth reading.

AI Chat Assistant:

While reading a PDF, students can tap the "Ask AI" button to ask questions based on the opened content. The screen displays a chat-style interface with AI responses shown instantly.

Teacher Upload Panel:

Teachers can upload study materials using a simple form with file selection, material type, and semester. Upload status (Pending/Approved/Rejected) is visible directly on the screen.

Admin Review Panel:

Admins receive a clear list of pending uploads and can approve or reject files with a single action. The interface shows file details and teacher information for verification.

Quiz Interface:

Students can attempt quizzes with multiple-choice questions. After submission, the system displays results and performance summaries in a clean, easy-to-read format.

Download Management:

Downloaded files appear in a dedicated section inside the app, allowing offline access while ensuring files remain within internal storage for security.

User Feedback

Confirmation Messages:

Actions such as file upload, quiz submission, and AI response delivery show brief confirmation prompts to reassure users.

Visual Cues:

Labels, icons, and colored indicators help users identify material types, upload statuses, and quiz outcomes quickly.

Error Handling:

Input validation ensures users enter correct information—for example, preventing empty upload fields or invalid login details.

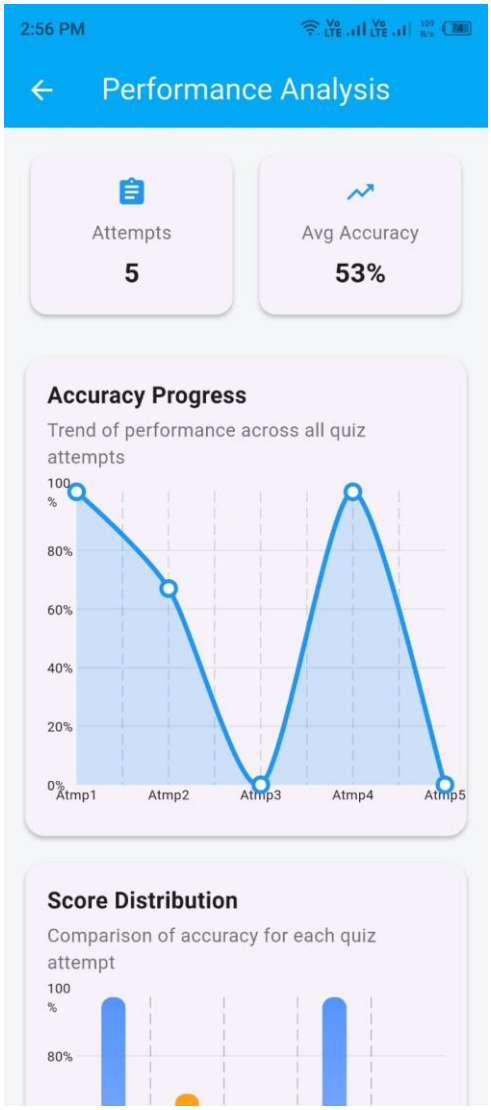
Notifications:

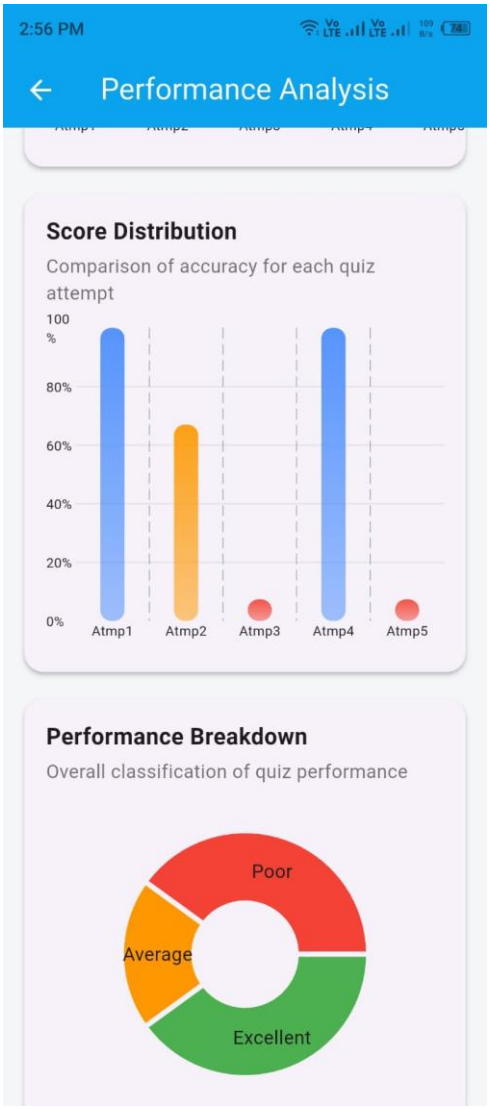
Push notifications alert students about new approved study material, quiz availability, or important system updates. Teachers and admins receive notifications for upload status changes.

3.5.1 Screen Images

Screen 1: Student Dashboard

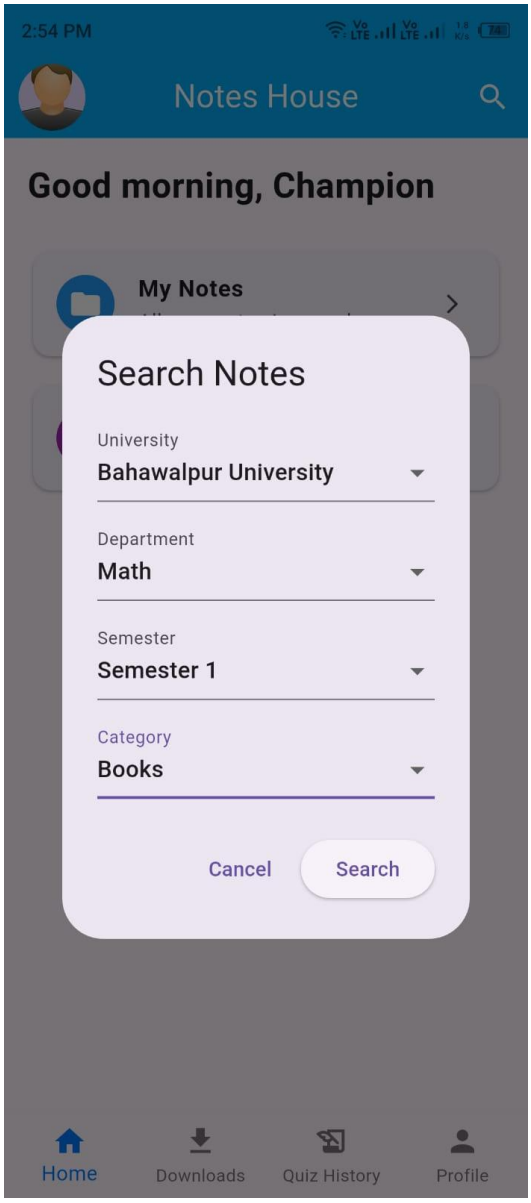
Purpose: To show the performance of the courses to the student for further improvement.





Screen 2: Searching Feature

Purpose: To navigate directly to the willing study materials



3.3.2 Screen Objects and Actions

Use Case 1: Student Views Quiz Performance Analysis

Screen Object	Description	Actions Associated
Attempts Card	Displays total number of quiz attempts	Auto-calculated from quiz history
Avg Accuracy Card	Shows average accuracy percentage	Updates after each quiz submission
Accuracy Progress Chart	Line graph showing performance trend across attempts	Automatically refreshes when new attempt is added
Score Distribution Chart	Bar chart comparing attempt-wise accuracy	Displays percentage of each attempt
Performance Breakdown Chart	Donut chart showing Excellent, Average, Poor categories	Categorizes scores based on defined ranges
Category Labels	Indicates performance level	Updates dynamically based on score
Back / Navigation Button	Returns to dashboard screen	On click: navigates to previous screen
Screen Title	Displays "Performance Analysis" heading	Static UI element

Use Case 2: Student Searches Study Material

Screen Object	Description	Actions Associated
Search Notes Title	Displays "Search Notes" heading	Static UI element
University Dropdown	Allows selection of university	On select: filters departments accordingly
Department Dropdown	Allows selection of department	On select: filters semesters
Semester Dropdown	Allows selection of semester	On select: filters available categories
Category Dropdown	Allows selection of material type (Books, Notes, Past Papers)	On select: prepares filtered query
Cancel Button	Closes search dialog	On click: returns to previous screen
Search Button	Initiates search process	On click: fetches filtered study material from database
Bottom Navigation Bar	Provides navigation to Home, Downloads, Quiz History, Profile	On click: navigates to selected screen

3.5 Design Decisions

The design of the Notes House system was guided by important decisions focused on scalability, maintainability, security, and usability. Below are the key decisions made during the system design**:**

1. Object-Oriented Design (OOD)

Decision:

The system was developed using an object-oriented design approach.

Justification:

Each class (e.g., User, StudyMaterial, UploadRequest, Quiz, AIChatSession) represents a real-world entity.

Improves modularity and makes the system easier to manage and extend.

Supports reusability and clean separation of responsibilities.

2. MVC (Model-View-Controller) Architecture

Decision:

The system architecture follows the MVC pattern.

Justification:

Ensures separation of concerns and organized development.

Model:

Manages core data such as users, study materials, quizzes, and uploads.

View:

Displays user interface screens such as dashboard, PDF reader, AI chat, and quiz screens.

Controller:

Handles user actions like searching notes, submitting quizzes, uploading files, and interacting with AI.

This architecture is suitable for mobile applications that require dynamic data handling and role-based access.

3.6 Summary

This chapter explained the system design and user interface models of the Notes House application. The design focused on modular architecture (MVC), structured data management, and smooth user interaction. The system ensures secure access, organized study material handling, quiz analysis, and AI-based assistance—aligning with the project goal of providing a centralized academic support platform for university students.

Chapter 4

Implementation

4. Implementation

This chapter discusses the implementation details of the Notes House project. The source code is not included here; instead, the core functionalities of the system are explained in pseudocode form.
All modules are implemented following proper coding standards to ensure clean structure, maintainability, and scalability. General Coding Standards & Guidelines are provided in Appendix D.

4.1 Algorithm

In Notes House, simple logic is used to implement the key features:

1. Study Material Search and Display

- The app receives the selected filters (University, Department, Semester, Category).
- It checks whether all required fields are selected.
- The system retrieves only approved study material from the database.
- Filtered results are displayed to the students.

If no material is found, a "No Results Found" message is shown.

2. Quiz Evaluation and Result Calculation

- The student attempts a quiz and submits answers.
- The system compares selected answers with correct answers.
- The total score and accuracy percentage are calculated.
- The result is saved in the database.
- Performance charts are updated accordingly.

3. AI Chat Processing

- The student enters a question in the AI chatbot.
- If a PDF is open, relevant text is extracted.
- The query (and extracted text if available) is sent to the AI service.
- The response is received and displayed to the user.
- Chat history is saved for future reference.

4. Teacher Upload and Admin Approval

- The teacher uploads a study file.
- The system validates file type and size.
- The file is stored with status "Pending".
- Admin reviews the file.
- If approved, the material becomes visible to students.
- If rejected, the teacher is notified.

Table 5-1 Example of Algorithm

Algorithm 1: Study Material Search Component
Input: University, Department, Semester, Category
Output: Filtered list of approved study materials
1: valid ← true 2: IF (University = NULL OR Department = NULL OR Semester = NULL OR Category = NULL) 3: valid ← false 4: DISPLAY "All fields are required" 5: END IF 6: IF valid = true THEN 7: materials ← FETCH FROM Database 8: FOR each item IN materials DO 9: IF (item.university = University AND 10: item.department = Department AND 11: item.semester = Semester AND 12: item.category = Category AND 13: item.status = "Approved") 14: ADD item TO result_list 15: END IF 16: END FOR 17: DISPLAY result_list 18: END IF 19: END

4.2 External APIs/SDKs

Describe the third-party APIs/SDKs used in the project implementation in the following table.

Few examples of APIs are provided in the table.

Table 5-1 Details of APIs used in the project

API Name	**Description of API**	**Purpose of Usage**	**API Endpoint / Function / Class Used**
Firebase Authentication (Current Version)	Provides secure user authentication and role-based login system.	To manage login, signup, and user access control (Student, Teacher, Admin).	authenticateUser()
Firebase Firestore (Current Version)	Cloud-based NoSQL database for storing structured data.	To store users, study materials, quizzes, results, and chat history.	fetchMaterials(), saveQuizResult(), getQuizData()
Firebase Storage (Current Version)	Cloud storage service for file upload and retrieval.	To upload and retrieve study material files (PDFs).	uploadFile(), downloadFile()
Firebase Cloud Messaging (FCM)	Sends push notifications to user devices.	To notify users about approvals, uploads, and updates.	sendNotification()
AI API Service	External AI service for generating responses.	To process user questions and generate answers in chatbot.	processAIQuery()

4.3 Code Repository

To manage the project effectively, we use Git as the primary version control system. All project files, including source code, documentation, UI designs, and testing reports, are maintained in a centralized repository.

1. **Git Repository Link:** https://github.com/HAS0786/Notes_House_FYP

4.3.1 Metrics to Track

2. **Commits:** Track updates and feature implementations in the project.
3. **Branches:** Separate branches for each module (e.g., Study Material Module, Quiz Module, AI Chat Module, Upload & Admin Module).
4. **Pull Requests:** Used for reviewing and merging new features into the main branch.
5. **Issues:** Track bugs, feature enhancements, and assigned tasks.
6. **Contributors:** Monitor individual team member contributions.
7. **Code Reviews:** Ensure coding standards, readability, and consistency across modules.

4.4 Summary

This chapter explained how Notes House is implemented using structured logic for study material management, quiz evaluation, AI chatbot interaction, and upload approval workflow. It also discussed the use of Firebase services for authentication, database management, file storage, and notifications. Git is used for version control and team collaboration, ensuring organized development and proper tracking of project progress.This implementation approach connects the system's academic support features with practical technical solutions to achieve the project objectives defined in earlier chapters.

Chapter 5

Testing and Evaluation

5. Introduction

To ensure proper verification and validation of the Notes House system, software testing is conducted. Testing helps confirm that all modules function correctly and meet the defined project requirements.

Both automated and manual test cases can be used for validation. Test cases should follow best practices such as:

- Keep tests independent so each test runs separately.
- Use meaningful assertions to clearly verify expected outcomes.
- Manage test data properly using setup and cleanup procedures.
- Avoid hardcoding values by using variables and configurations.
- Ensure tests are repeatable and do not fail due to saved states.

5.1 Unit Testing (UT)

Unit Testing verifies that individual components of the Notes House app work correctly. Each module is tested separately to ensure proper functionality.

Modules tested individually include:

- Study Material Search
- Quiz Evaluation Logic
- AI Chat Processing
- File Upload Validation
- Admin Approval Workflow

5.2 Functional Testing (FT)

Functional Testing ensures that each feature of Notes House meets the specified requirements. This includes testing:

1. **Study Material Search:**
   Does the system correctly filter materials based on selected university, department, semester, and category?
2. **Quiz Module:**
   Does it calculate scores accurately and display performance analysis charts?
3. **AI Chatbot:**
   Does it generate relevant responses based on user questions and opened PDFs?
4. **Upload and Approval Process:**
   Does the system properly manage teacher uploads and admin approvals?

5.3 Integration Testing (IT)

Integration Testing checks how different modules of Notes House work together as a complete system. It includes testing interactions between:

1. Study Material Search and Database (to ensure correct retrieval of approved files).
2. Quiz Module and Performance Analysis (to verify results are saved and displayed correctly).
3. AI Chat Module and PDF Reader (to confirm queries use relevant document content).
4. Upload Module and Admin Panel (to ensure approval status updates properly).

5.4 Performance Testing (PT)

Performance Testing evaluates how well Notes House handles different scenarios, such as:

• **Response Time:**
How quickly does the app load study materials, quiz data, and AI responses after user interaction?

• **Stability:**
Does the app remain stable during continuous usage such as multiple quiz attempts or repeated AI queries?

• **Reliability:**
Can the system handle multiple users searching notes, uploading files, and accessing quizzes without crashing?

• **Database Performance:**
Does the system retrieve and store data efficiently when the number of users and materials increases?

Tool Used: Android Profiler (Android Studio)
• Monitors CPU usage, memory consumption, and network activity.
• Tracks performance during file uploads, quiz submissions, and AI API calls.

Table 1 -Testcase

Field	Description
Testcase ID	Unique identifier of the testcase (e.g., UT_01, FT_02)
Requirement ID	Requirement ID of the feature to be tested
Title	Feature name or requirement to be tested in brief
Description	Description of the feature to be tested
Objective	Why this test case is needed
Driver / Precondition	Setup required before test execution
Test Steps	Step-by-step instructions to solve the test
Inputs	Input vales to be provided during testing
Expected Result	Expected outcome of the test case
Actual Result	Actual observed outcome after execution
Remarks	Test status (Pass/Fail)

5.5 Summary

This chapter discussed the testing and evaluation of the Notes House system. Testing ensures that each module, including Study Material Search, Quiz Module, AI Chatbot, Upload & Approval Workflow, and Notification System, performs as expected and integrates properly with other components.

Unit, Functional, Integration, and Performance testing were conducted to confirm system reliability, accuracy, and stability under different usage scenarios. This chapter ensures that Notes House meets the defined project objectives and operates effectively for students, teachers, and administrators.

Chapter 6

System Conversion

6. Introduction

System conversion is the process of shifting from an existing manual or scattered system to a new centralized digital system. In Notes House, this involves moving from traditional sharing methods (WhatsApp groups, physical notes, random PDFs) to a structured mobile application that manages study materials, quizzes, and AI assistance in one platform.

6.1 Conversion Method

For Notes House, the chosen conversion method is Phased Conversion.

Why Phased Conversion?

1. The system will be implemented in stages instead of launching all features at once.
2. First, the Study Material Search and Viewing module will be deployed and tested.
3. Next, the Quiz Module will be introduced and its evaluation system verified.
4. Then, the AI Chatbot feature will be integrated and monitored.
5. Finally, the Teacher Upload and Admin Approval workflow will be fully activated.
6. This approach reduces risk and allows testing and improvements at each stage.

6.2 Deployment

The deployment process includes building the application and preparing it for operational use. Since this is an academic project, deployment will be done in a controlled environment.

Deployment Steps:

1. **Build the Application**: Compile the Flutter project and generate the APK file.
2. **Firebase Configuration**: Configure Firebase Authentication, Firestore, and Storage.
3. **Backend Setup**: Connect the AI API and ensure database collections are structured properly.
4. **Install Application**: Install the APK on test devices or emulator.
5. **Testing**: Verify that all features (Search Notes, Quiz Module, AI Chat, Upload & Admin Approval) function correctly.

6.2.1 Data Conversion

Since Notes House is a new system and does not replace an existing automated platform, the data conversion process mainly involves initializing the database and setting up initial sample data.

Steps for Data Conversion:

1. **Data Initialization:**
   Create initial collections in Firebase such as Users, StudyMaterials, Quizzes, and UploadRequests.
2. **Sample Data Entry:**
   Add sample study materials, quiz questions, and user accounts (Student, Teacher, Admin) for testing purposes.
3. **Data Backup:**
   Export Firebase data to a secure backup file to prevent data loss.
4. **Data Restoration:**
   Restore the backup to verify that data can be recovered without errors.

6.3 Post Deployment Testing

After deploying Notes House, testing is performed to ensure that all features operate correctly:

1. **Study Material Search:**
   Select different filters (University, Department, Semester, Category) and verify that correct materials are displayed.
2. **Quiz Module:**
   Attempt quizzes and confirm that scores, accuracy, and performance charts are calculated properly.
3. **AI Chatbot:**
   Ask questions with and without an opened PDF and verify that responses are generated correctly.
4. **Upload & Admin Approval:**
   Upload study material as a teacher and confirm that admin approval updates visibility status correctly.

6.4 Challenges

Some challenges encountered during deployment and their solutions:

Challenge	Solution
Study materials not displaying after deployment	Verified Firebase configuration, checked database rules, and corrected file paths in storage.
AI responses not generating properly	Rechecked API integration and fixed request/response handling logic.
Upload approval status not updating	Debugged Firestore queries and ensured correct status field updates.
Notification delays	Adjusted Firebase Cloud Messaging settings for real-time updates.
Performance slowdown with large files	Optimized file loading and reduced unnecessary data fetching.

6.5 Summary

In this chapter, we discussed how Notes House was deployed using the Phased Conversion method by gradually implementing core modules such as Study Material Search, Quiz Module, AI Chatbot, and Upload & Admin Approval workflow.
The application was built, configured with Firebase services, and tested to ensure proper functionality. Data initialization, backup, and post-deployment testing were also performed. Key challenges, such as database configuration issues and notification delays, were resolved through proper debugging and optimization.
This structured deployment approach ensures system stability, scalability, and reliable performance for students, teachers, and administrators.

Chapter 7

Conclusion

7. Introduction

This chapter concludes the Notes House project, evaluates whether the defined objectives and goals were successfully achieved, and highlights possible future improvements.

7.1 Evaluation

This section evaluates whether the objectives defined at the beginning of the project have been successfully accomplished. A summary is presented below:

Objectives	Status
Provide centralized access to study materials	Completed
Implement semester-wise filtering and search	Completed
Develop quiz system with performance analysis	Completed
Integrate Al chatbot for academic assistance	Completed
Implement teacher upload and admin approval workflow	Completed
Ensure secure authentication and role-based access	Completed
Provide in-app PDF reader with Al integration	Completed
Enable notification system for updates and approvals	Completed

7.2 Traceability Matrix

The traceability matrix shows how each requirement was implemented in the system, where it was designed, and how it was tested.

Requirement ID	Requirement Description	Design Specification	Code / Module	Test ID
R1	Study Material Search and Filtering	Search & Filter Component	SearchController.dart	UT1
R2	Quiz Attempt and Evaluation	Quiz Management Module	QuizManager.dart	FT1
R3	AI Chatbot Assistance	AI Chat Module	AIChatController.dart	IT1
R4	Teacher Upload & Admin Approval	Upload & Review Workflow	UploadService.dart	UT2
R5	Secure Authentication & Role-Based Access	Authentication Module	AuthService.dart	PT1
R6	In-App PDF Reader with AI Integration	PDF Reader Component	PDFController.dart	FT2
R7	Notification System for Updates	Notification Module	NotificationService.dart	IT2

7.3 Conclusion

In conclusion, Notes House successfully achieved its main objectives of providing a centralized platform for study materials, quizzes, and AI-based academic assistance through a user-friendly mobile application.

The system allows students to search and access semester-wise notes, attempt quizzes with performance analysis, and interact with an AI chatbot for better understanding of study content. Teachers can upload materials, and admins can manage and approve content securely.

All defined objectives were completed, and the system was tested to ensure functionality, performance, and data security. Notes House proves to be a reliable academic support platform for university students across Pakistan.

7.4 Future Work

Possible improvements and future improvements for Notes House include:

1. **Advanced Analytics:**
   Provide detailed performance reports and subject-wise progress tracking for students.
2. **AI-Based Personalized Recommendations:**
   Suggest study materials and quizzes based on student performance.
3. **Multi-University Expansion:**
   Expand the system to include more universities and departments nationwide.
4. **Offline Access Enhancement:**
   Improve offline support for downloaded materials.
5. **Web Version Development:**
   Develop a web-based version for access through desktop browsers.

Appendix-A Use Case Description(Fully Dressed Format)

The Table A-1 below indicates a comprehensive use case template Table
A- 1Show the detail use case template and example

Use Case ID	UC-1
Use Case Name	Student Sign Up / Login
Actors	Student
Description	Allows a student to create a new account or log into the Notes House system to access study materials, quizzes, and AI chatbot features.
Trigger	Student clicks on the “Register” or “Login” button on the app home screen.
Preconditions	Student must have a valid email (edu email if required). App must be installed and internet connection available.
Postconditions	Student account is created successfully OR student is logged in and redirected to the dashboard.
Normal Flow	1. Student opens the app and selects “Register” or “Login”. 2. System displays form (email, password, name if signing up). 3. Student enters required information. 4. System validates credentials. 5. Student is redirected to dashboard.
Alternative Flows	Invalid or missing information: System displays error message and asks user to correct details.
Business Rules	Email must be unique. Password must meet minimum security requirements.
Assumptions	Student has basic knowledge of using a mobile app and stable internet access.
Use Case ID	UC-2
Actors	Teacher
Description	Allows a new teacher to create an account in the Notes House system using a verified educational email address.
Trigger	Teacher clicks on the “Register” option and selects Teacher account type.
Preconditions	Teacher must have a valid educational (.edu) email address. App must be installed and internet connection available.
Postconditions	Teacher account is created after verification. Teacher is redirected to login screen or dashboard.
Normal Flow	1. Teacher opens the app and selects “Register.” 2. Teacher chooses “Teacher” role. 3. System displays sign-up form (name, edu email, password, department). 4. Teacher fills the form and submits. 5. System verifies educational domain. 6. Account is created and confirmation shown.
Alternative Flows	Invalid or non-edu email: System displays error and requests valid educational email.
Business Rules	Email must belong to approved educational domain. Password must meet security requirements.
Assumptions	Teacher has institutional email access and basic digital literacy.

Use Case ID	UC-3
Use Case Name	View Study Materials & Quizzes
Actors	Student
Description	Allows the student to browse, view, and access available study materials and quizzes.
Trigger	Student selects “My Notes”, “Downloads”, or “Quizzes”.
Preconditions	Student must be logged in. Study materials must be available.
Postconditions	Selected notes displayed or downloaded. Quiz screen opened if selected.
Normal Flow	1. Student logs in. 2. Navigates to section. 3. System retrieves data. 4. Materials displayed. 5. Student opens note or starts quiz.
Alternative Flows	No data available: “No materials found” message. Network issue: Error and retry.
Business Rules	Students can only view approved materials. Downloaded files remain inside app storage only.
Assumptions	Internet available for new data. Offline access for downloaded files.
Use Case ID	UC-4
Use Case Name	Upload Study Materials
Actors	Teacher
Description	Allows teacher to upload study materials for admin review.
Trigger	Teacher selects “Upload Notes”.
Preconditions	Teacher logged in. Verified edu email. PDF format required.
Postconditions	File uploaded and sent for admin approval.
Normal Flow	1. Teacher selects upload. 2. System shows form. 3. Teacher selects PDF. 4. Clicks submit. 5. System uploads to Firebase. 6. Saved in Pending Uploads. 7. Confirmation shown.
Alternative Flows	Invalid file format. Network upload failure.
Business Rules	Only verified teachers can upload. Only approved content visible to students.
Assumptions	Stable internet and valid study material.
Use Case ID	UC-5
Use Case Name	Approve or Reject Uploaded Materials
Actors	Admin
Description	Allows admin to review and approve or reject uploaded materials.
Trigger	Admin opens “Pending Uploads”.
Preconditions	Admin logged in. Pending uploads exist.
Postconditions	Status updated to Approved or Rejected. Teacher notified.
Normal Flow	1. Admin logs in. 2. Opens pending uploads. 3. Reviews file. 4. Clicks Approve/Reject. 5. System updates database. 6. Notification sent.
Alternative Flows	Policy violation: Reject with reason. System error: Retry option.
Business Rules	Only admin can approve/reject. Only approved notes visible.
Assumptions	Admin reviews responsibly and notifications enabled.
Use Case ID	UC-6
Use Case Name	Attempt Quiz
Actors	Student
Description	Allows student to attempt quiz and view performance analysis.
Trigger	Student selects quiz.
Preconditions	Student logged in. Quiz approved.
Postconditions	Score calculated and saved. Performance displayed.
Normal Flow	1. Student opens quiz. 2. Selects answers. 3. Submits. 4. System calculates score. 5. Displays results. 6. Saves in database.
Alternative Flows	Incomplete submission confirmation. Network issue during submission.
Business Rules	Each question has one correct answer. Score stored securely.

Assumptions	Stable internet and quiz exists.
Use Case ID	UC-7
Use Case Name	AI Chatbot Interaction
Actors	Student
Description	Allows student to ask academic questions using AI Chatbot.
Trigger	Student clicks “AI Chat”.
Preconditions	Student logged in. Internet available.
Postconditions	AI response displayed. Chat history saved.
Normal Flow	1. Student opens AI Chat. 2. Enters question. 3. System sends to AI service. 4. AI responds. 5. Response displayed and saved.
Alternative Flows	API/network failure. Empty question prompt.
Business Rules	AI must remain academic. Chat history stored securely.
Assumptions	AI service operational.
Use Case ID	UC-8
Use Case Name	Manage Profile & Settings
Actors	Student, Teacher, Admin
Description	Allows users to update profile and manage settings.
Trigger	User selects “Profile” or “Settings”.
Preconditions	User logged in.
Postconditions	Updated info saved successfully.
Normal Flow	1. User opens profile. 2. Edits details. 3. Clicks save. 4. System validates and updates database. 5. Confirmation shown.
Alternative Flows	Invalid input. Password mismatch.
Business Rules	Email cannot be changed. Password must meet security rules.
Assumptions	User provides accurate info and has internet.

Appendix-B General Coding Standards & Guidelines

Technology Stack

Frontend: Flutter (Dart)
Backend Services: Firebase (Authentication, Firestore, Storage, FCM), Node.js , Express.js
Database: Firebase Firestore
Storage: Firebase Storage
AI Integration: External AI API Service

1. Naming Conventions

- Use camelCase for variables and method names
  (e.g., studentName, fetchMaterials(), calculateScore())
- Use PascalCase for class names
  (e.g., UserModel, StudyMaterial, QuizManager, AIChatController)
- Constants should be written in UPPER_SNAKE_CASE
  (e.g., MAX_QUIZ_ATTEMPTS, DEFAULT_TIMEOUT)
- Naming should clearly reflect the scope and purpose of the variable or method.

2. Variable and Field Declaration

- Use final for immutable variables and constants.
- Private class fields must be declared using private keyword.
- Avoid global variables unless absolutely necessary.
- Keep state variables inside relevant controllers or models.

3. Function and Method Naming

- Method names must be action-oriented and descriptive
  (e.g., authenticateUser(), uploadStudyMaterial(), processAIQuery()).
- Method names should describe what the function does, not how it does it.
- Keep methods short and focused on a single responsibility.

4. Commenting Standards

- Use proper documentation comments for classes and methods.

Example:

```
/// Calculates quiz score based on selected answers.
int calculateQuizScore(List answers) {
  // Business logic
}
```

- Avoid commenting obvious code.
- Comments should explain why a certain logic is implemented, not just what it does.

5. Code Structure & Indentation

- Use consistent 4-space indentation.
- Always enclose control structures in braces.

Example:

```
if (password.isEmpty) {
   showErrorMessage();
}
```

- Separate logical blocks with line spacing for better readability.
- Keep UI code, business logic, and data handling separated according to MVC/layered structure.

6. Project and Folder Naming

- Follow a modular folder structure:
- **screens/** – UI screens (LoginScreen, DashboardScreen, QuizScreen)
- **models/** – Data models (UserModel, StudyMaterial, QuizModel)
- **controllers/** – Business logic controllers (AuthController, QuizController, AIChatController)
- **services/** – Firebase and API services (AuthService, StorageService, NotificationService)
- **widgets/** – Reusable UI components
- **utils/** – Helper functions and constants
- File names should match class names (e.g., quiz_controller.dart, user_model.dart).

7. Class & File Design

- Each class must be defined in its own file.
- Use meaningful and descriptive class names.

- Follow **Single Responsibility Principle (SRP)**:
  Each class should handle one specific task only (e.g., QuizController handles quiz logic only).

8. Code Organization

- Group related logic into folders/modules:
- auth/ → login and registration logic
- materials/ → study material search and retrieval
- quiz/ → quiz handling and result evaluation
- ai/ → AI chatbot integration
- admin/ → upload approval workflow
- Avoid large monolithic files. Split features into smaller reusable modules.

9. Scope & Access Modifiers

- Use private variables (prefix with _ in Dart) by default.
- Expose only necessary public methods.
- Avoid exposing internal logic outside the class.
- Example:
- final String _apiKey;

10. Cohesion & Coupling

- Maintain **high cohesion**: Related functions should remain inside the same module.
- Maintain **low coupling**: Modules should interact through clearly defined services and interfaces.
- Controllers should not directly access UI elements.

11. DRY Principle (Don’t Repeat Yourself)

- Avoid duplicate logic.
- Extract common functions into utility classes or services.
- Break large functions into smaller reusable methods.
- Reuse methods instead of copying similar blocks of code.

12. Exception Handling

- Use structured try-catch blocks to handle runtime errors:
- try {

```
 await firestore.saveQuizResult(result);
} catch (e) {
  print("Error saving quiz result: $e");
}
```

- Do not ignore exceptions silently.
- Always log or handle errors properly.

13. Logging Standards

- Use debugPrint() or structured logging consistently.
- Tag logs by module (e.g., "QuizController", "AuthService").
- Avoid logging sensitive data such as passwords or tokens.

14. Extensibility & Maintainability

- Use service-based architecture for scalability.
- Design controllers and models in a way that new features (e.g., new quiz types, new AI tools) can be added without major modifications.
- Keep Firebase logic separated from UI logic to support future web version expansion.

Appendix-C

This appendix presents a visual and descriptive overview of the Notes House application prototype. It highlights the main interfaces and user interactions to provide a clear understanding of the app's layout, navigation flow, and overall user experience.

C.1 Tools Used

The prototype was developed using:

1. **Flutter (Dart)** – for UI design and screen development
2. **Firebase** – for backend integration (Authentication, Firestore, Storage)

C.2 Key Screens in the Prototype

1. Student Dashboard

1. Components:
1.1. Search Notes button
1.2. Quick access to Downloads
1.3. Quiz section
1.4. AI Chat shortcut
1.5. Bottom navigation bar (Home, Downloads, Quiz History, Profile)

2. Functionality:
Allows students to quickly access study materials, attempt quizzes, view downloads, and use the AI chatbot.

2. Study Material Search Screen

1. Components:
1.1. University dropdown
1.2. Department dropdown
1.3. Semester dropdown
1.4. Category dropdown (Notes, Books, Past Papers)
1.5. Search and Cancel buttons

2. Functionality:
Enables students to filter and retrieve approved study materials based on selected criteria.

3. In-App PDF Reader Screen

1. Components:
1.1. PDF viewing area
1.2. Zoom and scroll controls
1.3. Page navigation controls
1.4. "Ask AI" button

2. Functionality:
Allows students to read study material within the app and ask AI-based questions directly from the opened document.

4. Quiz Interface Screen

1. Components:
1.1. Question display area
1.2. Multiple-choice options
1.3. Submit button
1.4. Timer (if applicable)

2. Functionality:
Allows students to attempt quizzes and receive immediate score and performance analysis.

5. Teacher Upload Panel

1. Components:
1.1. File selection option
1.2. Subject and semester selection
1.3. Upload button
1.4. Upload status indicator

2. Functionality:
Allows teachers to upload study materials for admin review and approval.

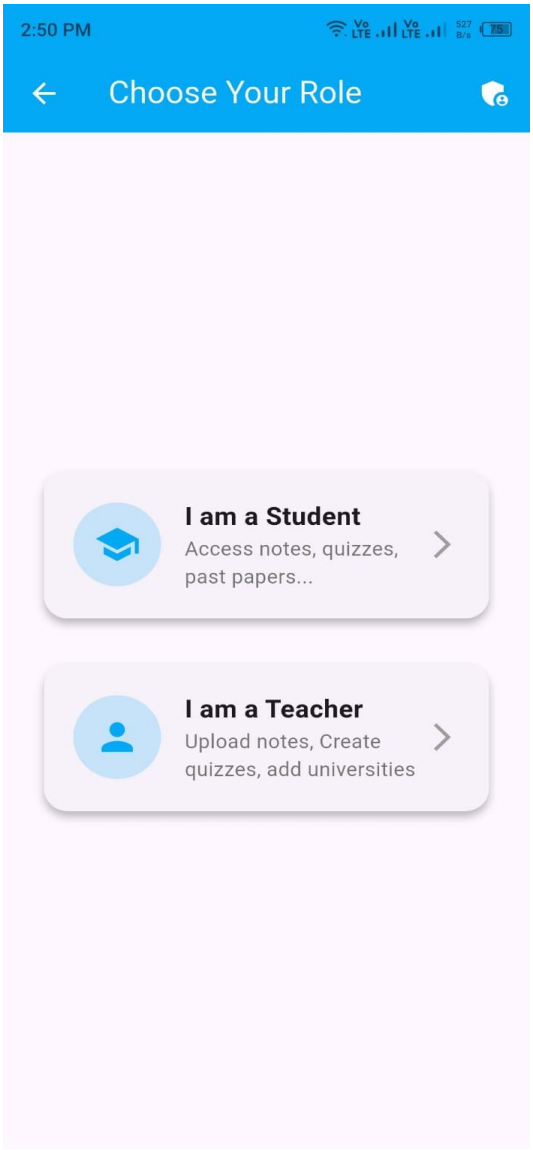
6. Admin Review Panel

1. Components:
1.1. List of pending uploads
1.2. File preview option
1.3. Approve and Reject buttons
1.4. Status indicator

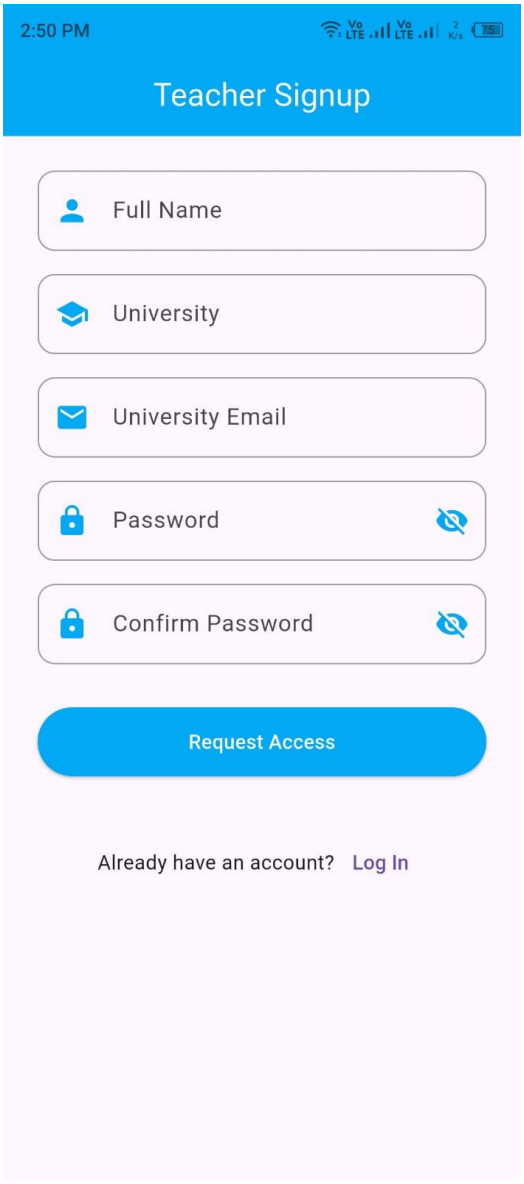
2. Functionality:
Enables admin to review uploaded materials and control their visibility in the system.

C.3 Prototype Screenshots:

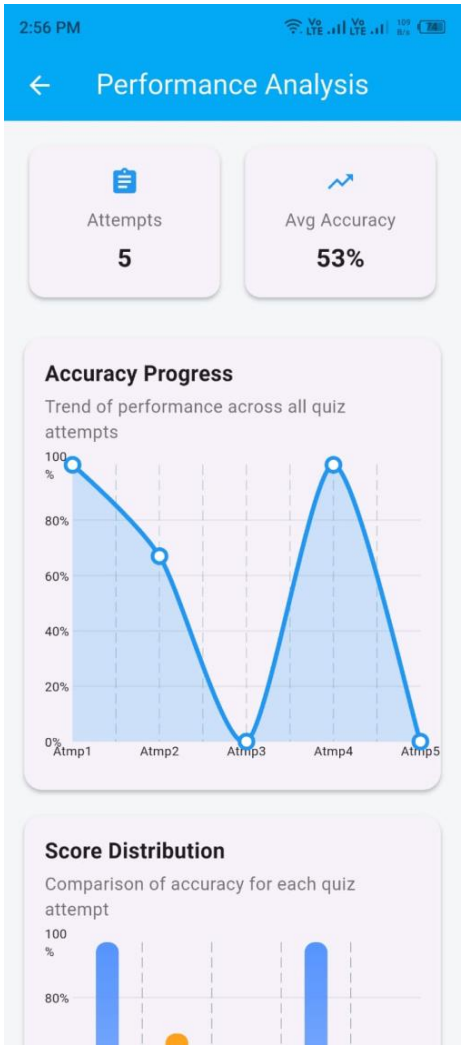
S1: Role:

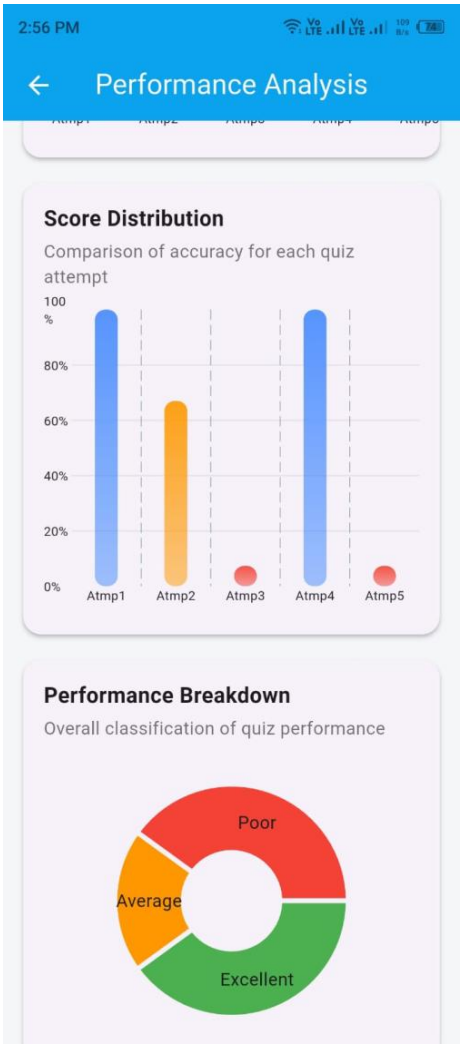


S2: Sign Up:

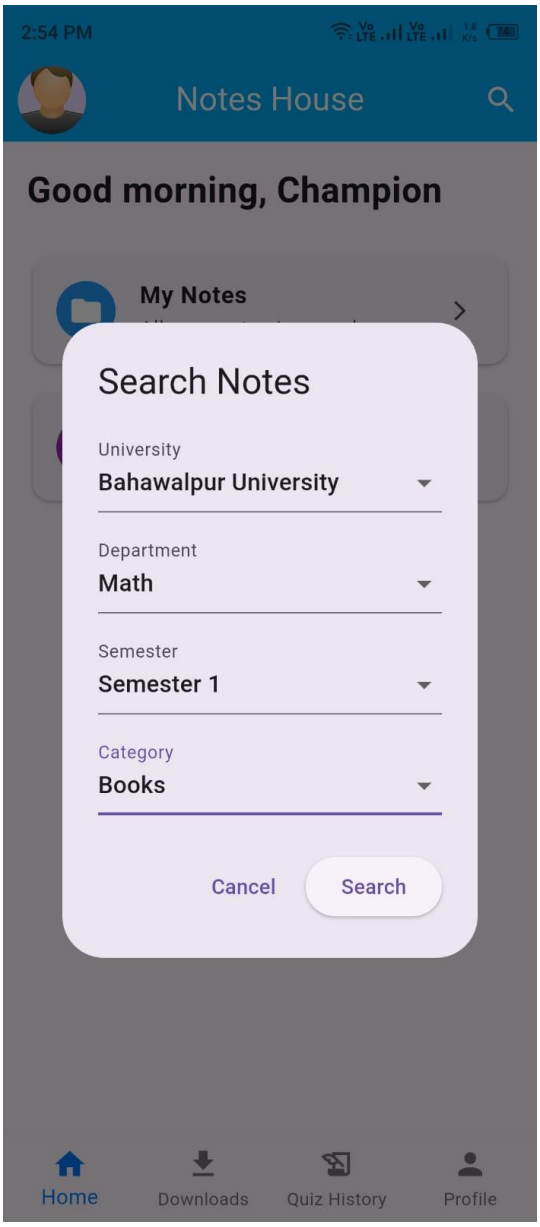

2:50 PM
Teacher Signup
Full Name
University
University Email
Password
Confirm Password
Request Access
Already have an account? Log In

S3: Dashboard:





S4: Search Function:



S5: Select University